NBA Expected Field-Goal Percentage — a SEMMA Project

Author: Meron Tesfandrias · **Course:** CMPE 255 · **Repo:** MeronDT/CMPE-255-Data-Mining-process · **App:** cmpe-255-data-mining-process-nba-expected-fg.streamlit.app

Data: NBA shot-chart detail (stats.nba.com), compiled by shufinskiy/nba_data (Apache-2.0; also on Kaggle). Every regular-season field-goal attempt with court coordinates, shot type, game clock and outcome. **Population:** five seasons, 2019-20 → 2023-24 (1,031,740 shots). **Out-of-time check:** 2024-25 (219,527 shots).

SEMMA (SAS Institute) organises the *technical* core of data mining into five steps. Each step below maps to the **SAS Enterprise Miner nodes** that implement it, so the same flow can be rebuilt in SAS (see the appendix).

Step	Purpose	Enterprise Miner nodes used here
Sample	draw a representative, manageable sample; partition it	Input Data · Sample · Data Partition
Explore	understand distributions, relationships, anomalies	StatExplore · MultiPlot / Graph Explore · Variable Selection · Cluster
Modify	create, select, transform variables	Drop · Transform Variables · Replacement · Interactive Binning · Impute
Model	fit competing predictive models	Regression · Decision Tree · Neural Network · Gradient Boosting · Ensemble
Assess	compare models, check reliability, score new data	Model Comparison · Cutoff · Score · Segment Profile

Headline result: a gradient-boosting model that estimates the make probability of any shot from where and how it is taken. It beats both the zone-average baseline and a stronger zone × shot-family baseline on every partition and on the unseen 2024-25 season (log loss 0.637 vs 0.647 and 0.658). It is calibrated to within 1.5 percentage points in every decile and within 2.1 points for every shot zone and shot family (except backcourt heaves). Most of the gain over location alone comes from the scorer's shot-type label (A.1). Comparing players' actual FG% with the xFG% of their shots separates shot *making* from shot *selection*. The gap persists from season to season (r = 0.70, 95 % CI 0.60–0.78, 131 players), though less than raw FG% does (0.86). So it is best read as shot-making plus context the model cannot see, not as pure skill.

How to run: *Runtime → Run all* (≈ 4–6 min on free Colab). Every figure, table, the model and the app are written to `artifacts/`.

Project context (pre-SEMMA)

SEMMA assumes the problem is already defined — unlike CRISP-DM it has no Business Understanding phase — so the problem is stated briefly here, to justify later choices.

- **Question:** given *where* and *how* a shot is taken (location, shot type, game clock), what is the probability it goes in? This is **expected field-goal percentage (xFG%)** — a measure of **shot quality**, independent of who shoots.
- **Why it matters:** raw FG% mixes two things: *which* shots a player takes and *how well* he makes them. A center who only dunks has a high FG% without being a great shooter. Comparing actual FG% with the xFG% of the same shots separates **shot selection** from **shot making** — the same idea as moving from box-score stats to advanced metrics when judging MVP candidates.
- **Target:** `SHOT_MADE_FLAG` (1 = made). **Unit:** one field-goal attempt.
- **Success criteria:** the champion model must
  1. beat a **zone-average baseline** (league FG% for the shot's official court zone) in validation **log loss**. A stronger **zone × shot-family** baseline is also reported, so the gain is not just "the model knows the shot type";
  2. be **calibrated** (predicted ≈ actual within each probability band, and within each shot zone and shot family);
  3. hold its performance on the **unseen 2024-25 season**.

⚙️ Setup & configuration

In [1]:
```
import json, platform, sys, tarfile, time, urllib.request, warnings
from datetime import datetime
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", 40)
plt.rcParams["figure.figsize"] = (8, 4.5)
print(f"python {platform.python_version()} | numpy {np.__version__} | pandas {pd
```

```
python 3.13.18 | numpy 2.5.3 | pandas 3.0.5 | scikit-learn 1.9.1
```

In [2]:
```
# ============================ CONFIGURATION ============================
URL_TEMPLATE = "https://github.com/shufinskiy/nba_data/raw/main/datasets/shotdet
POPULATION_SEASONS = [2019, 2020, 2021, 2022, 2023]   # file year = season start
OUT_OF_TIME_SEASON = 2024                              # 2024-25, scored only in
TARGET       = "SHOT_MADE_FLAG"
SAMPLE_SIZE  = 200_000          # shots drawn from the population (justified by
PARTITION    = (0.6, 0.2, 0.2)  # train / validate / test, split by GAME so shot
RANDOM_STATE = 42
# ======================================================================

ART = Path("artifacts"); DATA, FIG, REP, MOD, APP = (ART / p for p in ("data", "
for p in (DATA, FIG, REP, MOD, APP):
    p.mkdir(parents=True, exist_ok=True)

def savefig(name):
```

```
    plt.tight_layout(); plt.savefig(FIG / f"{name}.png", dpi=120, bbox_inches="t

def draw_court(ax, color="#555", lw=1.2):
    """NBA half court in shot-chart units (tenths of feet, hoop at 0,0)."""
    from matplotlib.patches import Circle, Rectangle, Arc
    for p in [Circle((0, 0), 7.5, fill=False), Rectangle((-30, -7.5), 60, 0), Re
              Arc((0, 142.5), 120, 120, theta1=0, theta2=180), Arc((0, 0), 80, 8
              Rectangle((-220, -47.5), 0, 140), Rectangle((220, -47.5), 0, 140),
              Rectangle((-250, -47.5), 500, 470, fill=False)]:
        p.set_edgecolor(color); p.set_linewidth(lw); ax.add_patch(p)
    ax.set_xlim(-250, 250); ax.set_ylim(-47.5, 422.5); ax.set_aspect("equal"); a
```

S — Sample

Goal: extract a portion of the data that is large enough to contain the significant information, yet small enough to process quickly — and partition it for honest model assessment.

S.1 Input Data

In [3]:

```
def load_season(season):
    path = DATA / f"shotdetail_{season}.csv"
    if not path.exists():                                    # download once, then
        archive = DATA / f"shotdetail_{season}.tar.xz"
        urllib.request.urlretrieve(URL_TEMPLATE.format(season=season), archive)
        with tarfile.open(archive, "r:xz") as t:
            t.extractall(DATA)
        archive.unlink()
    d = pd.read_csv(path, dtype={c: "category" for c in ["GRID_TYPE", "PLAYER_NA
                                                         "SHOT_TYPE", "SHOT_ZONE
    d.insert(0, "SEASON", f"{season}-{str(season + 1)[-2:]}")
    return d

t0 = time.time()
population = pd.concat([load_season(s) for s in POPULATION_SEASONS], ignore_inde
for c in population.select_dtypes("object").columns:
    population[c] = population[c].astype("category")
print(f"Population: {len(population):,} shots x {population.shape[1]} columns, {
      f"({time.time() - t0:.0f}s, {population.memory_usage(deep=True).sum() / 1e
display(population.groupby("SEASON", observed=True).agg(shots=(TARGET, "size"),
population.head(3)
```

```
Population: 1,031,740 shots x 25 columns, 5,829 games (2s, 121 MB)
```

SEASON	shots	games	fg_pct
2019-20	188116	1059	0.460
2020-21	190983	1080	0.466
2021-22	216722	1230	0.461
2022-23	217218	1230	0.475
2023-24	218701	1230	0.474

Out[3]:

	SEASON	GRID_TYPE	GAME_ID	GAME_EVENT_ID	PLAYER_ID	PLAYER_NAME	TEA
0	2019-20	Shot Chart Detail	21900014	9	1629631	De'Andre Hunter	16106
1	2019-20	Shot Chart Detail	21900014	11	203458	Alex Len	16106
2	2019-20	Shot Chart Detail	21900014	14	1629027	Trae Young	16106

The 2019-20 (COVID suspension and bubble) and 2020-21 (72-game) seasons have fewer games, but shooting barely differs between seasons (FG% 46.0–47.5 %), so pooling all five seasons is reasonable. Rising FG% in 2022-23 and 2023-24 is worth remembering for the out-of-time test.

S.2 Sample

In [4]:
```
# Proportional stratified sample: preserves each season's share and its make/mis
frac = SAMPLE_SIZE / len(population)
sampled = population.groupby(["SEASON", TARGET], observed=True, group_keys=False
remainder = population.drop(sampled.index)          # the shots NOT drawn - an i
sample = sampled.reset_index(drop=True)
print(f"Sample: {len(sample):,} shots ({len(sample) / len(population):.1%}); per

# Representativeness. FG% and season shares match BY DESIGN (they are the strata
# The real test compares the sample with the unsampled remainder on variables th
from scipy.stats import ks_2samp, chi2_contingency
def chi2_p(col, top=20):
    keep = remainder[col].value_counts().index[:top]
    pool = lambda s: s.astype(str).where(s.isin(keep), "other").value_counts()
    return chi2_contingency(pd.concat([pool(sample[col]), pool(remainder[col])],
check = pd.DataFrame([
    {"variable": "FG% (stratum - matches by design)", "sample": sample[TARGET].m
    {"variable": "shot distance (mean ft)", "sample": sample["SHOT_DISTANCE"].me
     "test": "KS", "p_value": ks_2samp(sample["SHOT_DISTANCE"], remainder["SHOT_
    {"variable": "LOC_X (mean)", "sample": sample["LOC_X"].mean(), "remainder":
     "test": "KS", "p_value": ks_2samp(sample["LOC_X"], remainder["LOC_X"]).pval
    {"variable": "3PT share", "sample": (sample["SHOT_TYPE"] == "3PT Field Goal"
     "remainder": (remainder["SHOT_TYPE"] == "3PT Field Goal").mean(), "test": "
    {"variable": "ACTION_TYPE mix (top 20 + other)", "sample": np.nan, "remainde
    {"variable": "SHOT_ZONE_BASIC mix", "sample": np.nan, "remainder": np.nan, "
```

```
    {"variable": "SHOT_ZONE_AREA mix", "sample": np.nan, "remainder": np.nan, "t
    {"variable": "overtime share", "sample": sample["PERIOD"].gt(4).mean(), "rem
     "test": "chi-square", "p_value": chi2_contingency(pd.crosstab(np.r_[np.zero
                                                                   np.r_[sample
]).set_index("variable").round(4)
check.to_csv(REP / "sample_representativeness.csv"); display(check)
```

```
Sample: 199,999 shots (19.4%); per-stratum rounding gives 1 fewer than configured
```

variable	sample	remainder	test	p_value
FG% (stratum - matches by design)	0.4677	0.4677	-	NaN
shot distance (mean ft)	13.5769	13.5667	KS	0.9897
LOC_X (mean)	-1.6838	-1.9570	KS	0.4519
3PT share	0.3926	0.3915	chi-square	0.3725
ACTION_TYPE mix (top 20 + other)	NaN	NaN	chi-square	0.8468
SHOT_ZONE_BASIC mix	NaN	NaN	chi-square	0.6956
SHOT_ZONE_AREA mix	NaN	NaN	chi-square	0.5748
overtime share	0.0059	0.0060	chi-square	0.8573

Representativeness. FG% and season shares match the population **by design**, because they are the strata, so they are not evidence. The real test compares the sample with the **unsampled remainder** (832k shots, an independent group) on variables that were *not* stratified on. No test finds a difference:

- shot distance (KS p = 0.99) and horizontal location (p = 0.45);
- 3-point share (χ^2 p = 0.37) and overtime share (p = 0.86);
- the mix of shot types (`ACTION_TYPE` , p = 0.85) and of court zones (p = 0.70 and 0.57).

Because the sampling is *proportional*, the base rate is not distorted, which matters because the model must output real probabilities. Per-stratum rounding gives 199,999 shots instead of 200,000.

S.3 Data Partition

In [5]:
```
from sklearn.model_selection import GroupShuffleSplit

# Split by GAME_ID: shots from the same game never sit in two partitions. This g
# correlation only; player-level correlation (the same shooter in all partitions
# shooter identity is not a model input, and the out-of-time season is the real
tr_frac, va_frac, te_frac = PARTITION
idx_train, idx_rest = next(GroupShuffleSplit(1, test_size=va_frac + te_frac, ran
                           .split(sample, groups=sample["GAME_ID"]))
rest = sample.iloc[idx_rest]
idx_val, idx_test = next(GroupShuffleSplit(1, test_size=te_frac / (va_frac + te_
                         .split(rest, groups=rest["GAME_ID"]))
train, valid, test = sample.iloc[idx_train], rest.iloc[idx_val], rest.iloc[idx_t
```

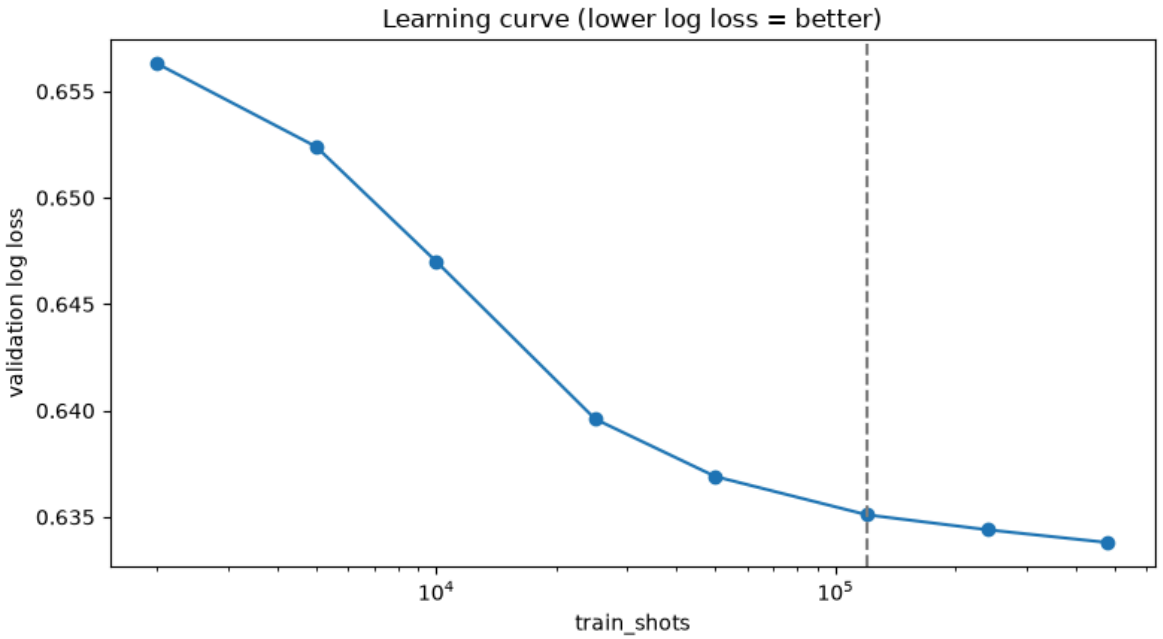
```
partitions = pd.DataFrame({p: {"shots": len(d), "games": d["GAME_ID"].nunique(),
                               **{f"share {s}": round((d["SEASON"] == s).mean(),
                           for p, d in [("train", train), ("validate", valid), (
partitions.to_csv(REP / "data_partition.csv"); display(partitions)
g_tr, g_va, g_te = set(train.GAME_ID), set(valid.GAME_ID), set(test.GAME_ID)
assert not (g_tr & g_va) and not (g_tr & g_te) and not (g_va & g_te), "a game ap
```

	shots	games	FG%	share 2019-20	share 2020-21	share 2021-22	share 2022-23	share 2023-24
train	120014.0	3497.0	0.4690	0.176	0.183	0.215	0.211	0.215
validate	40064.0	1166.0	0.4675	0.181	0.195	0.193	0.219	0.211
test	39921.0	1166.0	0.4639	0.203	0.180	0.213	0.200	0.204

In [6]:
```
# Is SAMPLE_SIZE big enough? Learning curve: validation log loss vs. number of t
# beyond the 120k training partition come from population games that are in neit
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import roc_auc_score, log_loss

quick = lambda d: d[["LOC_X", "LOC_Y", "PERIOD", "MINUTES_REMAINING", "SECONDS_R
    action=d["ACTION_TYPE"].cat.codes)
pool = population[~population["GAME_ID"].isin(g_va | g_te)]
curve = []
for n in [2_000, 5_000, 10_000, 25_000, 50_000, 120_000, 240_000, 480_000]:
    sub = pool.sample(n, random_state=RANDOM_STATE)
    m = HistGradientBoostingClassifier(max_iter=500, early_stopping=True, random
    p = m.predict_proba(quick(valid))[:, 1]
    curve.append({"train_shots": n, "val_auc": roc_auc_score(valid[TARGET], p),
curve = pd.DataFrame(curve).round(4); curve.to_csv(REP / "learning_curve.csv", i
fig, ax = plt.subplots()
curve.plot(x="train_shots", y="val_log_loss", marker="o", logx=True, legend=Fals
ax.axvline(len(train), c="gray", ls="--"); ax.set_ylabel("validation log loss");
```

	train_shots	val_auc	val_log_loss
0	2000	0.6338	0.6563
1	5000	0.6359	0.6524
2	10000	0.6393	0.6470
3	25000	0.6512	0.6396
4	50000	0.6580	0.6369
5	120000	0.6620	0.6351
6	240000	0.6636	0.6344
7	480000	0.6650	0.6338



Is the sample big enough? The learning curve now uses early stopping (so small samples are not just over-fitted). It extends past the 120k training partition with extra shots from population games that sit in neither validation nor test. Validation log loss falls quickly up to ~25k shots (0.656 → 0.640), then slowly: 0.6351 at 120k, 0.6344 at 240k, 0.6338 at 480k. The last 360k shots buy 0.0013. That is not nothing: it is about the size of the gap between the top models in A.1. So the 200k sample is enough to *compare* model families, which is SEMMA's argument for sampling. The model that is *deployed* is then refit on train + validate (160k shots) to use more of the data.

Partitioning. Train/validate/test = 60/20/20 **grouped by game** (3,497 / 1,166 / 1,166 games), so shots from one game never sit in two partitions. FG% is 46.9 / 46.8 / 46.4 %, and season shares per partition are within 3 points of each other. Grouping by game guards against **within-game** correlation only. The same players and teams still appear in all three partitions. That is acceptable because shooter and team identity are not inputs, and the out-of-time 2024-25 season (new games, a new year) is the real test of generalisation.

Output of Sample: a stratified, representative 200k-shot sample; three game-grouped partitions; plus the full 2024-25 season held back for an out-of-time check in Assess.

🔍 Critic review — Sample

Each phase was reviewed in three rounds with the expert-critic prompt below. Round 1 was run in the drafting session. Rounds 2 and 3 were run by fresh, independent Claude sessions that saw only the executed notebook, its figures and the app files. Full reviews and point-by-point responses: `critiques/round_1.md`, `round_2.md`, `round_3.md`.

> *You are a world-renowned authority on the SEMMA data mining methodology and SAS Enterprise Miner, a sought-after keynote speaker, and the author of several award-winning books on SEMMA and predictive modeling. Review the **Sample** phase of my NBA expected-field-goal-*

> *percentage project below. Check it against everything SEMMA's Sample step should accomplish (and the Enterprise Miner nodes that implement it). Focus especially on: whether the sample is demonstrably representative, whether the sample size is justified, whether the partitioning prevents leakage (grouping by game, out-of-time hold-out), and whether stratification choices are explained. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Learning curve uses a reduced feature set, so it only indicates the needed size; stratifying on the target needs justification; two atypical seasons.	Stated as indicative (flat after ~25k shots, so 200k has a wide margin); showed proportional stratification keeps FG% exactly; checked season FG% (46.0–47.5 %).
2	Representativeness evidence circular (stratified variables; KS against its own population); learning curve too short. (B overall)	Tests against the unsampled remainder on unstratified variables; learning curve to 480k shots; deployed model refit on 160k.
3	Sound; minor wording on the partition. (B+ overall)	No further change needed.

E — Explore

*Goal: search the data for trends, anomalies and relationships to gain understanding and ideas for Modify and Model. Explore uses the **training partition** only.*

E.1 StatExplore

In [7]:
```
ID_COLS = ["GAME_ID", "GAME_EVENT_ID", "PLAYER_ID", "TEAM_ID", "GAME_DATE"]
display(train.drop(columns=ID_COLS).describe(include="number").T.round(2))
cat_summary = pd.DataFrame({c: {"levels": train[c].nunique(), "most common": tra
                                "top share": round(train[c].value_counts(normali
                           for c in train.select_dtypes("category").columns}).T
display(cat_summary)
print(f"Missing values in the training partition: {int(train.isna().sum().sum())

# Constant columns and outcome-revealing columns
constant = [c for c in train.columns if train[c].nunique() <= 1]
leak = pd.crosstab(train["EVENT_TYPE"], train[TARGET])
print("Constant columns:", constant); print("EVENT_TYPE vs target:"); display(le

# Anomalies: shots from beyond half court, behind the backboard, or logged in th
anomalies = pd.DataFrame([{"anomaly": name, "shots": int(m.sum()), "share_%": ro
                           for name, m in {"LOC_Y > 400 (beyond half court)": tra
                                           "LOC_Y < -7.5 (behind the backboard)":
                                           "SHOT_DISTANCE > 40 ft": train["SHOT_D
```

```
                                                "zone = Backcourt": train["SHOT_ZONE_B
anomalies.to_csv(REP / "explore_anomalies.csv", index=False); display(anomalies)
```

	count	mean	std	min	25%	50%	75%	max
PERIOD	120014.0	2.49	1.13	1.0	1.0	2.0	3.0	7.0
MINUTES_REMAINING	120014.0	5.38	3.45	0.0	2.0	5.0	8.0	12.0
SECONDS_REMAINING	120014.0	28.87	17.43	0.0	14.0	29.0	44.0	59.0
SHOT_DISTANCE	120014.0	13.55	10.58	0.0	2.0	13.0	25.0	81.0
LOC_X	120014.0	-1.61	112.82	-250.0	-48.0	0.0	45.0	250.0
LOC_Y	120014.0	95.72	95.06	-50.0	14.0	55.0	183.0	797.0
SHOT_ATTEMPTED_FLAG	120014.0	1.00	0.00	1.0	1.0	1.0	1.0	1.0
SHOT_MADE_FLAG	120014.0	0.47	0.50	0.0	0.0	0.0	1.0	1.0

	levels	most common	top share
SEASON	5	2023-24	0.215
GRID_TYPE	1	Shot Chart Detail	1.0
PLAYER_NAME	896	Jayson Tatum	0.008
TEAM_NAME	30	Orlando Magic	0.037
EVENT_TYPE	2	Missed Shot	0.531
ACTION_TYPE	48	Jump Shot	0.316
SHOT_TYPE	2	2PT Field Goal	0.609
SHOT_ZONE_BASIC	7	Restricted Area	0.301
SHOT_ZONE_AREA	6	Center(C)	0.567
SHOT_ZONE_RANGE	5	Less Than 8 ft.	0.41
HTM	30	IND	0.037
VTM	30	ORL	0.038

```
Missing values in the training partition: 0 (all 25 columns checked)
Constant columns: ['GRID_TYPE', 'SHOT_ATTEMPTED_FLAG']
EVENT_TYPE vs target:
```

SHOT_MADE_FLAG	**0**	**1**
EVENT_TYPE		
Made Shot	0	56283
Missed Shot	63731	0

	anomaly	shots	share_%	FG%
0	LOC_Y > 400 (beyond half court)	251	0.21	0.036
1	LOC_Y < -7.5 (behind the backboard)	2105	1.75	0.428
2	SHOT_DISTANCE > 40 ft	262	0.22	0.034
3	zone = Backcourt	229	0.19	0.026

Findings.

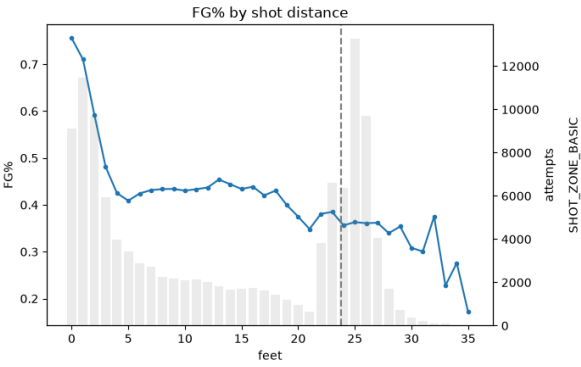
- `EVENT_TYPE` ("Made Shot"/"Missed Shot") matches the target perfectly (56,283 makes, 63,731 misses, zero disagreements). It **is the target in disguise** and must be dropped.
- `GRID_TYPE` and `SHOT_ATTEMPTED_FLAG` are constant, and there are no missing values in any of the 25 columns.
- Shots cluster at the rim (30 % in the restricted area) and beyond the arc (39 % threes); `Jump Shot` alone is 32 % of attempts.

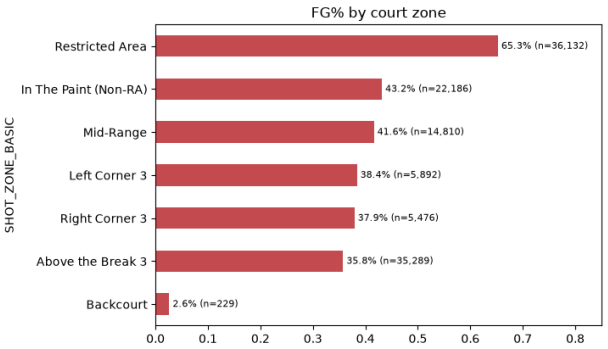
Anomalies:

- 251 shots come from beyond half court (FG 3.6 %) and 229 are logged in the backcourt zone (2.6 %). These are real end-of-quarter heaves, kept and flagged by the `heave` variable.
- 2,105 shots (1.75 %) have `LOC_Y` below the hoop line. They are baseline shots under and behind the rim (FG 42.8 %), also real.
- Only 2 of the 48 `ACTION_TYPE` labels are missing from some season (0.12 % of shots), so the label set is stable across seasons. `ACTION_TYPE` is assigned by the scorer after the play, so a missed dunk *could* be logged as a layup. Whether that happens cannot be checked without video, but its *consequence* can be measured: A.1 fits the champion without the shot-type label.

E.2 MultiPlot / Graph Explore

In [8]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))
by_ft = train[train["SHOT_DISTANCE"] <= 35].groupby("SHOT_DISTANCE")[TARGET].agg
axes[0].plot(by_ft.index, by_ft["mean"], marker="o", ms=3); axes[0].axvline(23.7
axes[0].set(title="FG% by shot distance", xlabel="feet", ylabel="FG%")
ax2 = axes[0].twinx(); ax2.bar(by_ft.index, by_ft["size"], alpha=.15, color="gra
zone = train.groupby("SHOT_ZONE_BASIC", observed=True)[TARGET].agg(["mean", "siz
zone["mean"].plot.barh(ax=axes[1], color="#C44E52", title="FG% by court zone")
for i, (m, n) in enumerate(zip(zone["mean"], zone["size"])):
    axes[1].text(m, i, f" {m:.1%} (n={n:,})", va="center", fontsize=8)
axes[1].set_xlim(0, .85); savefig("explore_distance_zone")
print("FG% at selected distances (feet):"); display(by_ft.loc[[0, 1, 2, 3, 5, 10
```

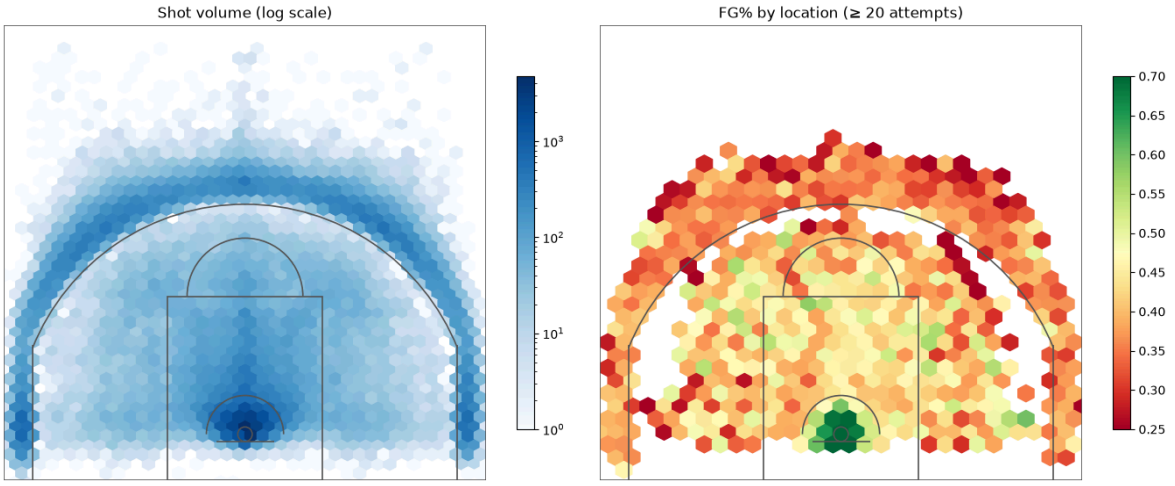




FG% at selected distances (feet):

SHOT_DISTANCE	0	1	2	3	5	10	15	20	24	26	30
FG%	0.756	0.71	0.592	0.481	0.409	0.43	0.434	0.375	0.356	0.361	0.308

In [9]:
```
# Shot chart: where shots are taken, and how often they go in
fig, axes = plt.subplots(1, 2, figsize=(14, 7))
h = train[train["LOC_Y"] < 400]
hb = axes[0].hexbin(h["LOC_X"], h["LOC_Y"], gridsize=40, bins="log", cmap="Blues
draw_court(axes[0]); axes[0].set_title("Shot volume (log scale)"); fig.colorbar(
hb2 = axes[1].hexbin(h["LOC_X"], h["LOC_Y"], C=h[TARGET], reduce_C_function=np.m
                     vmin=.25, vmax=.70, mincnt=20)
draw_court(axes[1]); axes[1].set_title("FG% by location (≥ 20 attempts)"); fig.c
savefig("explore_shot_charts")
```

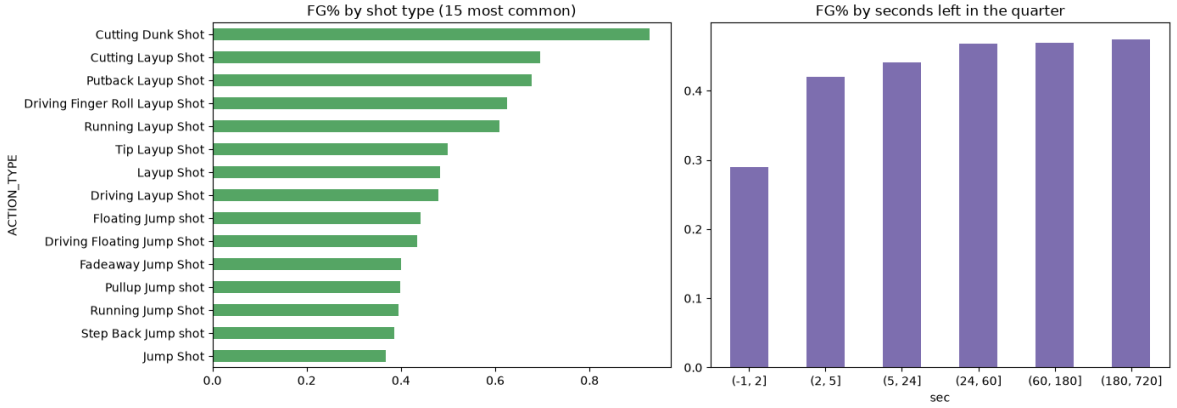


In [10]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
act = train.groupby("ACTION_TYPE", observed=True)[TARGET].agg(["mean", "size"]).
act["mean"].plot.barh(ax=axes[0], color="#55A868", title="FG% by shot type (15 m
clock = train.assign(sec=train["MINUTES_REMAINING"] * 60 + train["SECONDS_REMAIN
clock = clock[clock["PERIOD"] <= 4].groupby(pd.cut(clock["sec"], [-1, 2, 5, 24,
clock.plot.bar(ax=axes[1], rot=0, color="#8172B2", title="FG% by seconds left in
savefig("explore_action_clock")

# Is the end-of-quarter drop about the clock, or about distance (heaves)? Split
t2 = train.assign(sec=train["MINUTES_REMAINING"] * 60 + train["SECONDS_REMAINING
late = pd.crosstab(pd.cut(t2["sec"], [-1, 3, 24, 720], labels=["<= 3 s left", "4
                   pd.cut(t2["SHOT_DISTANCE"], [-1, 10, 23, 29, 100], labels=["0
                   values=t2[TARGET], aggfunc="mean").round(3)
late_n = pd.crosstab(pd.cut(t2["sec"], [-1, 3, 24, 720]), pd.cut(t2["SHOT_DISTAN
print("FG% by time left x distance (shot counts in the 30+ ft column:", late_n.i

# Is ACTION_TYPE labelled consistently across seasons? Labels that appear only i
```

```
label_seasons = train.groupby("ACTION_TYPE", observed=True)["SEASON"].nunique()
print(f"{(label_seasons < train['SEASON'].nunique()).sum()} of {len(label_season
      f"they cover {train['ACTION_TYPE'].isin(label_seasons[label_seasons < trai
```



FG% by time left x distance (shot counts in the 30+ ft column: [459, 29, 674])

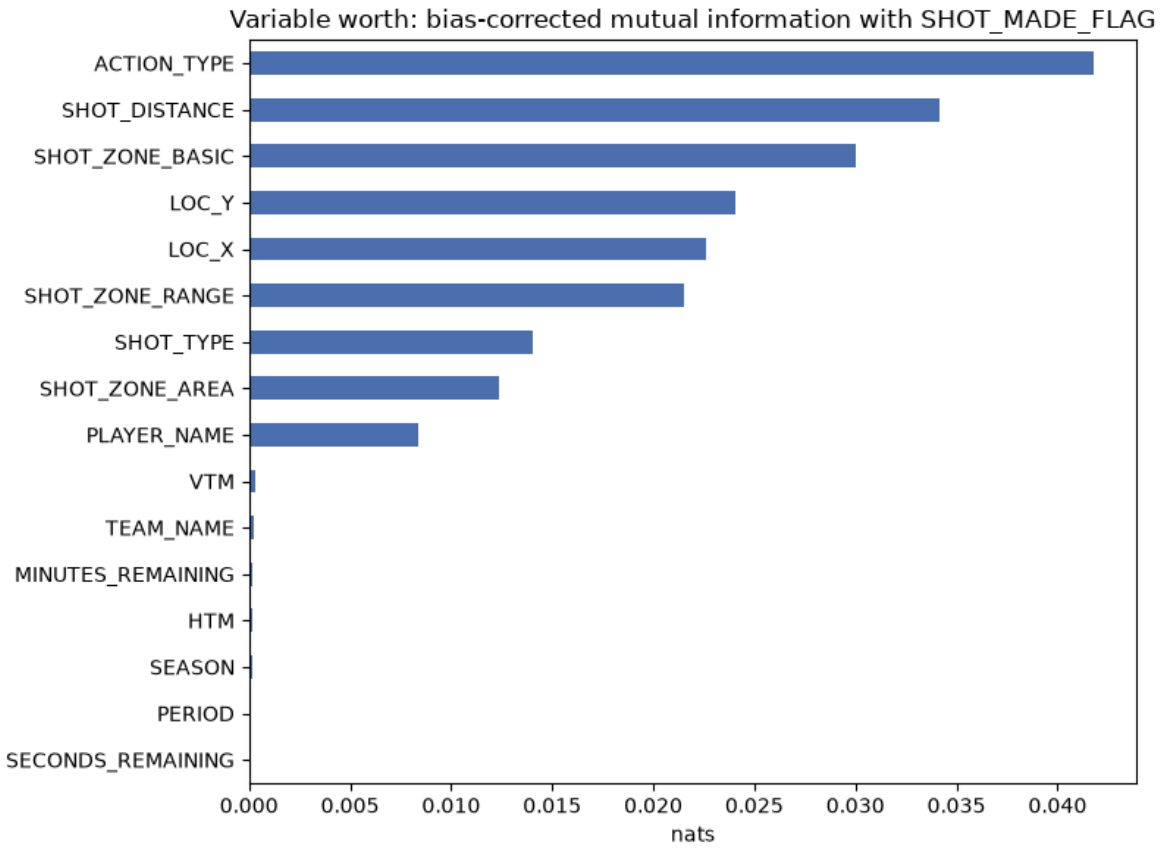
SHOT_DISTANCE	0-10 ft	11-23 ft	24-29 ft	30+ ft
sec				
<= 3 s left	0.489	0.335	0.230	0.081
4-24 s	0.558	0.362	0.329	0.172
> 24 s	0.574	0.413	0.363	0.329

2 of 48 action labels are missing from at least one season; they cover 0.12% of shots

E.3 Variable Selection (variable worth)

In [11]:
```
from sklearn.feature_selection import mutual_info_classif
cand = train.drop(columns=[TARGET, "EVENT_TYPE", "GRID_TYPE", "SHOT_ATTEMPTED_FL
s = cand.sample(30_000, random_state=RANDOM_STATE)
enc = s.apply(lambda c: c.cat.codes if hasattr(c, "cat") else c)
discrete = [hasattr(cand[c], "cat") or cand[c].nunique() < 20 for c in enc.colum
mi = pd.Series(mutual_info_classif(enc, train.loc[s.index, TARGET], discrete_fea
# Plug-in MI on a discrete variable with K levels is biased upward by about (K -
# without the correction a variable with hundreds of levels (PLAYER_NAME) looks
bias = pd.Series({c: (s[c].nunique() - 1) / (2 * len(s)) if d else 0.0 for c, d
worth = pd.DataFrame({"mutual_information": mi, "small_sample_bias": bias, "corr
                      "levels": s.nunique()}).sort_values("corrected_MI", ascend
worth.round(5).to_csv(REP / "variable_worth.csv"); display(worth.round(4))
fig, ax = plt.subplots(figsize=(8, 6))
worth["corrected_MI"].sort_values().plot.barh(ax=ax, color="#4C72B0", title="Var
ax.set_xlabel("nats"); savefig("explore_variable_worth")
```

	mutual_information	small_sample_bias	corrected_MI	levels
ACTION_TYPE	0.0426	0.0008	0.0418	48
SHOT_DISTANCE	0.0342	0.0000	0.0342	65
SHOT_ZONE_BASIC	0.0301	0.0001	0.0300	7
LOC_Y	0.0241	0.0000	0.0241	441
LOC_X	0.0226	0.0000	0.0226	498
SHOT_ZONE_RANGE	0.0216	0.0001	0.0215	5
SHOT_TYPE	0.0140	0.0000	0.0140	2
SHOT_ZONE_AREA	0.0125	0.0001	0.0124	6
PLAYER_NAME	0.0218	0.0134	0.0084	806
VTM	0.0008	0.0005	0.0003	30
TEAM_NAME	0.0007	0.0005	0.0003	30
MINUTES_REMAINING	0.0004	0.0002	0.0002	13
HTM	0.0007	0.0005	0.0002	30
SEASON	0.0002	0.0001	0.0001	5
PERIOD	0.0002	0.0001	0.0001	6
SECONDS_REMAINING	0.0000	0.0000	0.0000	60



Variable worth.

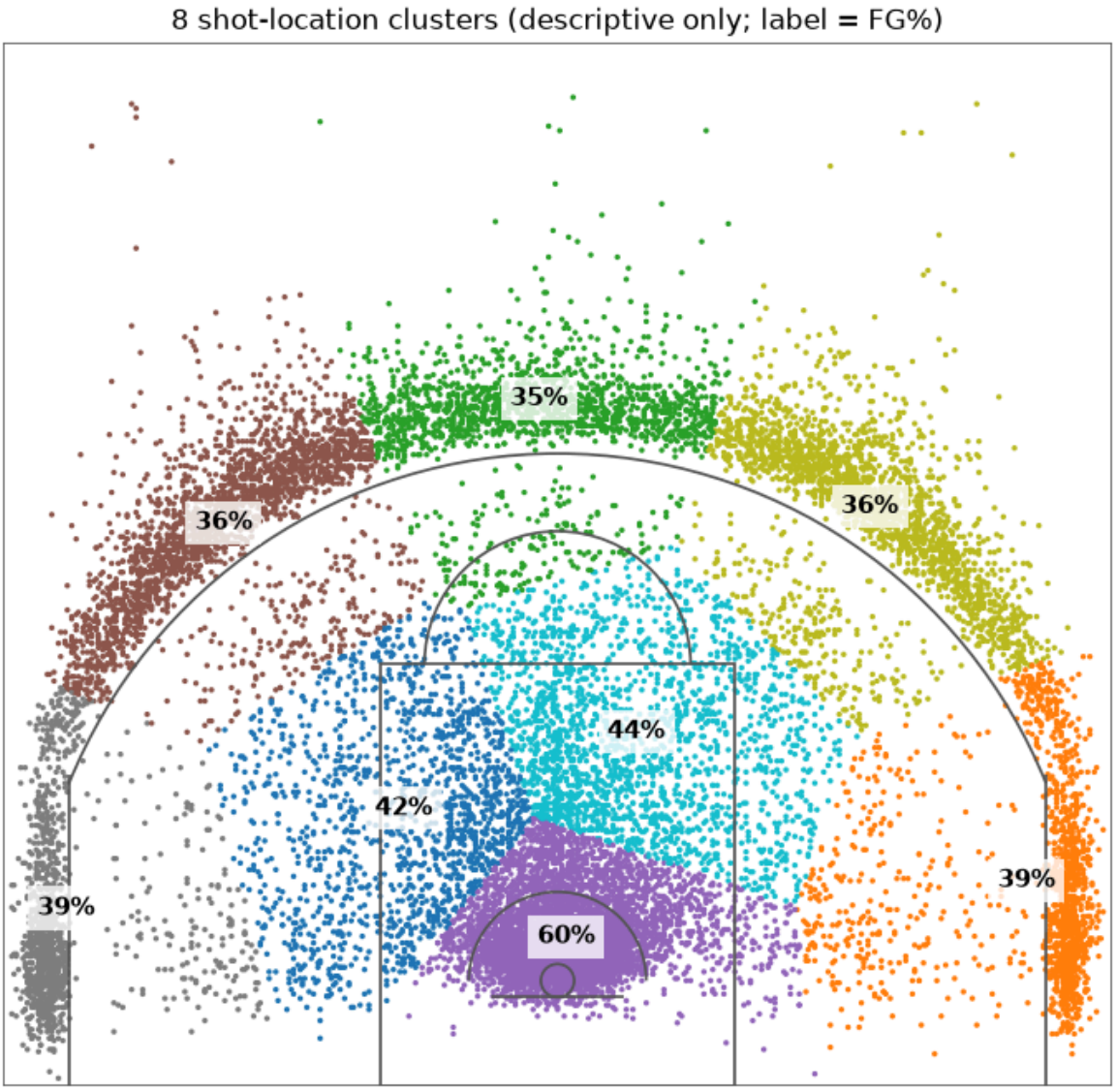
- Shot type (`ACTION_TYPE` , MI 0.042) and distance (`SHOT_DISTANCE` 0.034, zone 0.030, `LOC_Y` 0.024, `LOC_X` 0.023) carry the most information.
- Mutual information on a variable with many levels is biased upward. For `PLAYER_NAME` (806 levels in the sample) the bias is 0.013 of its raw 0.022, so after the correction shooter identity carries only **0.008**. It is excluded *by design* anyway: this model measures shot **quality**, and who made the shot is analysed afterwards (A.4) as shot **making**.
- Game-clock fields carry almost nothing on their own (≤ 0.0002) but matter in one situation, end-of-quarter heaves (E.2).
- Distance is represented several overlapping ways (distance, zone, range, location). Modify replaces them with one consistent geometry.

E.4 Cluster (shot-location segments)

In [12]:
```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=8, n_init=10, random_state=RANDOM_STATE).fit(train[["LOC_
seg = train.assign(cluster=km.labels_)
profile = seg.groupby("cluster")[TARGET].agg(attempts="size", fg_pct="mean").rou
profile["mean_distance_ft"] = seg.groupby("cluster")["SHOT_DISTANCE"].mean().rou
display(profile.sort_values("mean_distance_ft"))
fig, ax = plt.subplots(figsize=(7, 7)); h = seg[seg["LOC_Y"] < 400].sample(20_00
ax.scatter(h["LOC_X"], h["LOC_Y"], c=h["cluster"], cmap="tab10", s=2); draw_cour
for k, (x, y) in enumerate(km.cluster_centers_):
    ax.text(x, y, f"{profile.loc[k, 'fg_pct']:.0%}", ha="center", fontsize=11, w
            bbox=dict(facecolor="white", alpha=.8, edgecolor="none"))
ax.set_title("8 shot-location clusters (descriptive only; label = FG%)"); savefi

# Corner vs above-the-break threes at (nearly) the same distance: is it the angl
threes = train[train["SHOT_TYPE"] == "3PT Field Goal"].assign(corner=lambda d: d
matched = threes[threes["SHOT_DISTANCE"].between(22, 25)].pivot_table(index="SHO
matched.columns = [f"{a} {'corner' if b else 'above the break'}" for a, b in mat
print("3PT FG% by exact (integer) distance, corner vs above the break:"); displa
```

cluster	attempts	fg_pct	mean_distance_ft
3	46323	0.602	2.2
0	8576	0.421	10.5
7	11489	0.444	11.6
1	8680	0.386	21.5
5	7809	0.388	22.2
4	13820	0.365	25.2
6	13305	0.358	25.2
2	10012	0.347	25.8



3PT FG% by exact (integer) distance, corner vs above the break:

SHOT_DISTANCE	mean above the break	mean corner	size above the break	size corner
22	0.353	0.384	17	3441
23	0.394	0.385	109	6372
24	0.352	0.370	4893	1474
25	0.364	0.222	13180	81

Exploration findings

#	Finding (training partition)	Implication for Modify / Model
1	FG% falls fastest right at the rim: 76 % at 0 ft, 71 % at 1 ft, 59 % at 2 ft, 48 % at 3 ft, 41 % at 5 ft. It stays flat at about 43 % through mid-range (10–15 ft), drops to about 36 % at the arc, 31 % at 30 ft, 23 % at 33 ft and 17 % at 35 ft	strongly non-linear distance effect → **fine distance bins near the rim** plus a log distance for the regression; trees handle it natively

#	Finding (training partition)	Implication for Modify / Model
2	Corner threes go in more than above-the-break threes, partly because they are shorter (22–23 ft vs 24–25 ft). At the **same** integer distance of 24 ft the gap shrinks to 37.0 % vs 35.2 % (n = 1,474 vs 4,893)	a corner-three flag; the shot angle is tested in Model (it adds nothing measurable)
3	Shot **type** matters as much as location: dunks 89 %, layups 56 %, floaters 45 %, pull-ups 40 %, jump shots 37 %. Rare labels (e.g. "Running Alley Oop Dunk Shot") are too small to model alone	keep `ACTION_TYPE`; add a **shot family** so rare labels are pooled with shots of the same kind
4	Late-clock shots go in less **at every distance**: with ≤ 3 s left, 49 % (0–10 ft) and 8 % (30+ ft), against 57 % and 33 % with more than 24 s left	add the seconds left and a **heave** flag (≤ 3 s and ≥ 30 ft). No finding supports a separate clutch or overtime flag, so neither is used
5	K-means on location finds one rim cluster (60 % FG, 2 ft), two mid-range clusters (42–44 %), two corner/short-three clusters (≈ 39 %) and three above-the-break clusters (35–37 %). The clusters are descriptive only and are not used as inputs	consistent with finding 2
6	Anomalies: heaves from beyond half court and the backcourt (0.4 % of shots), baseline shots behind the hoop line (1.75 %)	keep: all real shots; the `heave` flag covers the first group
⚠️	`EVENT_TYPE` duplicates the target; `GRID_TYPE`, `SHOT_ATTEMPTED_FLAG` constant	drop

🔍 Critic review — Explore

Each phase was reviewed in three rounds with the expert-critic prompt below. Round 1 was run in the drafting session. Rounds 2 and 3 were run by fresh, independent Claude sessions that saw only the executed notebook, its figures and the app files. Full reviews and point-by-point responses: `critiques/round_1.md`, `round_2.md`, `round_3.md`.

> *You are a world-renowned authority on the SEMMA data mining methodology and SAS Enterprise Miner, a sought-after keynote speaker, and the author of several award-winning books on SEMMA and predictive modeling. Review the **Explore** phase of my NBA expected-field-goal-percentage project below. Check it against everything SEMMA's Explore step should accomplish (and the Enterprise Miner nodes that implement it). Focus especially on: whether exploration used only the training partition, whether findings are quantified and translated into concrete Modify actions, and whether leakage and redundant variables were detected. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Player identity has high variable worth but is dropped without argument; clusters are only descriptive.	Explained the shot-quality vs shot-making design choice; linked clusters to the angle/corner variables.
2	Variable-worth figure corrupted; plug-in MI inflated PLAYER_NAME; anomalies and redundancy not analysed.	Figure fixed; bias-corrected MI (PLAYER_NAME 0.022 → 0.008); anomaly table; label stability by season.
3	"Matched-distance" corner comparison not matched; ACTION_TYPE risk declared untestable; no redundancy matrix.	Comparison at exactly 24 ft (37.0 % vs 35.2 %); correlation matrix of engineered inputs; label risk measured by ablation in Model.

M — Modify

*Goal: create, select and transform variables to focus the model-building process. Every transformation is written as **score code** — a small Python module reused unchanged by the deployed app, the equivalent of Enterprise Miner's exported score code.*

M.1 Drop

In [13]:
```
DROP = {
    "EVENT_TYPE": "duplicates the target (Made/Missed Shot) - leakage",
    "GRID_TYPE": "constant", "SHOT_ATTEMPTED_FLAG": "constant (every row is an a
    "GAME_ID": "identifier (used only for partitioning)", "GAME_EVENT_ID": "iden
    "PLAYER_ID": "shooter identity - excluded by design (model measures shot qua
    "TEAM_ID": "team identity - excluded by design", "TEAM_NAME": "team identity
    "HTM": "home team code - home/away is context, but the app scores a shot wit
    "VTM": "visiting team code - see HTM",
    "SEASON": "season label - unseen seasons at scoring time",
    "SHOT_ZONE_BASIC": "derivable from LOC_X/LOC_Y - replaced by engineered geom
    "SHOT_ZONE_RANGE": "derivable from location", "SHOT_DISTANCE": "integer-roun
    "SHOT_TYPE": "2PT/3PT - recomputed from court geometry so the app needs only
}
drop_log = pd.DataFrame({"variable": list(DROP), "reason": list(DROP.values())})
drop_log.to_csv(REP / "modify_drop_log.csv", index=False); display(drop_log)
print("Inputs kept:", [c for c in train.columns if c not in DROP and c != TARGET
```

	variable	reason
0	EVENT_TYPE	duplicates the target (Made/Missed Shot) - lea...
1	GRID_TYPE	constant
2	SHOT_ATTEMPTED_FLAG	constant (every row is an attempt)
3	GAME_ID	identifier (used only for partitioning)
4	GAME_EVENT_ID	identifier
5	GAME_DATE	time index - would not generalise to new seasons
6	PLAYER_ID	shooter identity - excluded by design (model m...
7	PLAYER_NAME	shooter identity - excluded by design
8	TEAM_ID	team identity - excluded by design
9	TEAM_NAME	team identity - excluded by design
10	HTM	home team code - home/away is context, but the...
11	VTM	visiting team code - see HTM
12	SEASON	season label - unseen seasons at scoring time
13	SHOT_ZONE_BASIC	derivable from LOC_X/LOC_Y - replaced by engin...
14	SHOT_ZONE_AREA	derivable from location
15	SHOT_ZONE_RANGE	derivable from location
16	SHOT_DISTANCE	integer-rounded; replaced by exact distance fr...
17	SHOT_TYPE	2PT/3PT - recomputed from court geometry so th...

```
Inputs kept: ['PERIOD', 'MINUTES_REMAINING', 'SECONDS_REMAINING', 'ACTION_TYPE',
'LOC_X', 'LOC_Y']
```

M.2 Transform Variables · Replacement · Interactive Binning — as score code

In [14]:
```
SCORE_CODE = r"""
# Score code for the NBA xFG% model: the Modify step, shared by the notebook and
import numpy as np
import pandas as pd

RAW_INPUTS = ["LOC_X", "LOC_Y", "ACTION_TYPE", "PERIOD", "MINUTES_REMAINING", "S
NUMERIC = ["dist_ft", "log_dist", "abs_angle", "is_three", "is_corner_three", "s
CATEGORICAL = ["action", "action_family", "distance_bin"]
# finer bins near the rim, where FG% falls fastest (Explore finding 1)
BIN_EDGES = [-0.1, 1, 2, 3, 5, 10, 16, 23.75, 30, 100]
BIN_LABELS = ["0-1 ft", "1-2 ft", "2-3 ft", "3-5 ft", "5-10 ft", "10-16 ft", "16


def action_family(action):
    # Group the scorer's ACTION_TYPE labels into shot families, so rare labels (
    # are still recognised as what they are instead of being pooled with unrelat
    if pd.isna(action):
        return np.nan
```

```
    a = str(action).lower()
    for family, keys in [("dunk", ["dunk"]), ("tip/putback", ["tip", "putback"])
                         ("layup", ["layup", "finger roll"]), ("hook", ["hook"])
                         ("bank shot", ["bank"]), ("fadeaway/turnaround", ["fade
                         ("pull-up/step-back", ["pull", "step back"]), ("jump sh
        if any(k in a for k in keys):
            return family
    return "other"


def engineer(raw):
    # Transform raw shot records (RAW_INPUTS) into model inputs. Stateless, so i
    # Missing inputs stay missing (NaN) so the Impute step in the pipeline can h
    d = pd.DataFrame(raw).copy()
    num = lambda c: pd.to_numeric(d[c], errors="coerce")
    x, y = num("LOC_X"), num("LOC_Y")                                  # tenths of
    has_loc = x.notna() & y.notna()
    out = pd.DataFrame(index=d.index)
    out["dist_ft"] = np.hypot(x, y) / 10
    out["log_dist"] = np.log1p(out["dist_ft"])
    out["abs_angle"] = np.degrees(np.arctan2(x, y)).abs()              # 0 = strai
    corner = (x.abs() >= 220) & (y <= 92.5)
    out["is_corner_three"] = corner.astype(float).where(has_loc)
    out["is_three"] = (corner | (np.hypot(x, y) >= 237.5)).astype(float).where(h
    out["secs_left_period"] = num("MINUTES_REMAINING") * 60 + num("SECONDS_REMAI
    out["PERIOD"] = num("PERIOD")
    out["heave"] = ((out["secs_left_period"] <= 3) & (out["dist_ft"] >= 30)).ast
        out["secs_left_period"].notna() & has_loc)
    out["action"] = d["ACTION_TYPE"].astype(object).where(d["ACTION_TYPE"].notna
    out["action_family"] = out["action"].map(action_family)
    out["distance_bin"] = pd.cut(out["dist_ft"], BIN_EDGES, labels=BIN_LABELS).a
    out["distance_bin"] = out["distance_bin"].where(out["dist_ft"].notna(), np.n
    return out[NUMERIC + CATEGORICAL]


class AveragedModel:
    # Ensemble node: averages the predicted probabilities of already-fitted pipe
    def __init__(self, models):
        self.models = models
        self.classes_ = np.array([0, 1])

    def predict_proba(self, X):
        p = np.mean([m.predict_proba(X)[:, 1] for m in self.models], axis=0)
        return np.column_stack([1 - p, p])

    def predict(self, X):
        return (self.predict_proba(X)[:, 1] >= 0.5).astype(int)
"""
(APP / "shot_features.py").write_text(SCORE_CODE.lstrip(), encoding="utf-8")
if str(APP) not in sys.path:
    sys.path.insert(0, str(APP))
import importlib, shot_features
importlib.reload(shot_features)
from shot_features import engineer, AveragedModel, action_family, RAW_INPUTS, NU

agree = (engineer(train)["is_three"] == (train["SHOT_TYPE"] == "3PT Field Goal")
print(f"Geometry-based 3PT flag agrees with the official SHOT_TYPE on {agree:.2%
fam = train.assign(family=train["ACTION_TYPE"].map(action_family)).groupby("fami
display(fam.sort_values("FG_pct", ascending=False).round(3).T)
engineer(train.head()).join(train[["SHOT_TYPE", "SHOT_DISTANCE", TARGET]].head()
```

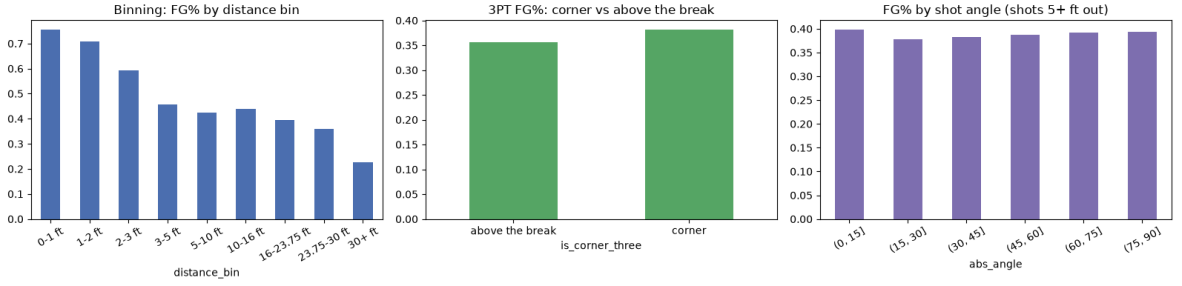
Geometry-based 3PT flag agrees with the official SHOT_TYPE on 99.95% of training shots

family	dunk	alley-oop	tip/putback	layup	bank shot	hook	floater	fadeaway/t
shots	6915.000	554.00	4378.000	27648.000	667.000	3597.000	9783.000	
FG_pct	0.891	0.63	0.577	0.556	0.552	0.483	0.453	

Out[14]:

	dist_ft	log_dist	abs_angle	is_three	is_corner_three	secs_left_period	PERIOD	h
0	26.482069	3.313534	30.147421	1.0	0.0	398	3	
1	26.126806	3.300522	42.052414	1.0	0.0	641	2	
2	24.647921	3.244463	48.289243	1.0	0.0	205	3	
4	26.447306	3.312268	31.706851	1.0	0.0	548	4	
5	19.089526	3.000199	30.191623	0.0	0.0	13	4	

In [15]:
```
# Did the transformations create signal? FG% by the new variables (training part
eng = engineer(train).assign(made=train[TARGET].to_numpy())
fig, axes = plt.subplots(1, 3, figsize=(16, 4))
eng.groupby("distance_bin")["made"].mean().reindex(shot_features.BIN_LABELS).plo
eng[eng["is_three"] == 1].groupby("is_corner_three")["made"].mean().rename({0: "
eng[eng["dist_ft"] >= 5].groupby(pd.cut(eng["abs_angle"], [0, 15, 30, 45, 60, 75
savefig("modify_new_variables")
print("Redundancy among the engineered numeric inputs (Pearson correlation, trai
display(eng[NUMERIC].corr().round(2))
```



Redundancy among the engineered numeric inputs (Pearson correlation, training partition):

	dist_ft	log_dist	abs_angle	is_three	is_corner_three	secs_left_period	PE
dist_ft	1.00	0.95	-0.05	0.88	0.29	-0.03	
log_dist	0.95	1.00	-0.04	0.76	0.28	-0.01	
abs_angle	-0.05	-0.04	1.00	0.04	0.45	0.00	
is_three	0.88	0.76	0.04	1.00	0.41	-0.03	
is_corner_three	0.29	0.28	0.45	0.41	1.00	-0.00	
secs_left_period	-0.03	-0.01	0.00	-0.03	-0.00	1.00	
PERIOD	0.01	0.00	-0.01	0.01	0.00	-0.02	
heave	0.18	0.09	-0.05	0.08	-0.02	-0.10	

M.3 Replacement & Impute (inside the model pipeline)

Rare `action` *labels (< 0.5 % of training shots) are* ***replaced*** *by a pooled "infrequent" level, but each keeps its* ***shot family*** *through* `action_family`*. The pooling applies to the raw label only; all 10 families and all distance bins are kept, and an unseen family or bin is simply ignored. The 19 pooled labels are 6 dunks, 5 bank shots, 3 layups, 3 hooks and 2 alley-oops, so a rare dunk is still scored as a dunk. Missing values (none in training, but possible in new data) stay missing through the score code and are* ***imputed*** *in the pipeline; the probes confirm that a shot with missing location, clock and shot type is scored (xFG 0.55, close to the league average), and an unseen label is still scored by its family ("Spinning Reverse Dunk Shot" at the rim: 0.92). Both steps are learned from the training partition only.*

Changes made after review:

- *finer distance bins at the rim (0–1, 1–2, 2–3, 3–5 ft) and a log distance, following finding 1;*
- *one angle representation (* `abs_angle` *) instead of two;*
- *the* `clutch` *and* `is_overtime` *flags removed, because no Explore finding supported them.*

Redundancy (table above): distance appears as `dist_ft`*,* `log_dist` *(r = 0.95),* `is_three` *(0.88) and the distance bins. That is harmless for the tree models. For the regression it means the coefficients are identified only by the L2 penalty, which is why they are read only for direction.*

In [16]:
```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer, OneHotEncoder, StandardSc
from sklearn.impute import SimpleImputer

def make_prep(numeric=NUMERIC, use_action=True):
    # Rare-level Replacement applies to the raw scorer label only; shot family a
    # (an unseen value is simply ignored rather than mapped to some pooled level
```

```
    impute_cat = lambda: SimpleImputer(strategy="constant", fill_value="missing"
    parts = [("num", Pipeline([("impute", SimpleImputer(strategy="median")), ("s
    if use_action:
        parts += [("act", Pipeline([("impute", impute_cat()), ("replace_rare", O

                  ("cat", Pipeline([("impute", impute_cat()), ("onehot", OneHotE
                   ["action_family", "distance_bin"])]
    else:
        parts += [("cat", Pipeline([("impute", impute_cat()), ("onehot", OneHotE
    return ColumnTransformer(parts)

prep_preview = make_prep().fit(engineer(train))
names = prep_preview.get_feature_names_out()
pooled = prep_preview.named_transformers_["act"].named_steps["replace_rare"].inf
print(f"{len(names)} model inputs after Replacement/one-hot. {len(pooled)} rare
      f"but each keeps its own shot family through action_family (all {engineer(
display(pd.Series({a: action_family(a) for a in pooled}).value_counts().to_frame
X_tr, X_va, X_te = (d[RAW_INPUTS] for d in (train, valid, test))
y_tr, y_va, y_te = (d[TARGET].to_numpy() for d in (train, valid, test))

# Probes for the Impute step and for unseen labels: they must be scored, and sen
probe = pd.DataFrame([{"LOC_X": np.nan, "LOC_Y": 50, "ACTION_TYPE": None, "PERIO
                      {"LOC_X": 0, "LOC_Y": 5, "ACTION_TYPE": "Spinning Reverse
                     index=["missing location, clock and shot type", "unseen lab
print("Probe rows engineered as:"); display(engineer(probe))
```

```
57 model inputs after Replacement/one-hot. 19 rare action labels are pooled, but
each keeps its own shot family through action_family (all 10 families are kept):
```

	dunk	bank shot	layup	hook	alley-oop
rare labels per family	6	5	3	3	2

```
Probe rows engineered as:
```

	dist_ft	log_dist	abs_angle	is_three	is_corner_three	secs_left_period	PERIOD
missing location, clock and shot type	NaN	NaN	NaN	NaN	NaN	NaN	2
unseen label, still a dunk at the rim	0.5	0.405465	0.0	0.0	0.0	360.0	1

🔍 Critic review — Modify

Each phase was reviewed in three rounds with the expert-critic prompt below. Round 1 was run in the drafting session. Rounds 2 and 3 were run by fresh, independent Claude sessions that saw only the executed notebook, its figures and the app files. Full reviews and point-by-point responses: `critiques/round_1.md`, `round_2.md`, `round_3.md`.

> *You are a world-renowned authority on the SEMMA data mining methodology and SAS Enterprise Miner, a sought-after keynote speaker, and*

> *the author of several award-winning books on SEMMA and predictive modeling. Review the **Modify** phase of my NBA expected-field-goal-percentage project below. Check it against everything SEMMA's Modify step should accomplish (and the Enterprise Miner nodes that implement it). Focus especially on: whether every transformation is justified by an Explore finding, whether anything is fitted on non-training data, whether dropped variables (leakage, identity, redundancy) are correctly reasoned, and whether the score code is reusable at deployment. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	`ACTION_TYPE` is coded by scorers and could reflect the outcome; verify the geometry-based 3PT flag.	Recorded as a limitation (standard in public xFG models); showed 99.95 % agreement of the 3PT flag.
2	Rare-level pooling merged dunks with bank shots; imputation never reached (casts crashed on NaN); unsupported flags.	Shot families; NaN-safe score code + probe; finer rim bins and log distance; clutch / overtime flags dropped.
3	Pooling also collapsed the alley-oop family; unseen inputs defaulted to it.	Pooling limited to the raw label; families and bins keep every level; probes check the predicted value.

M — Model

*Goal: let the software search for a combination of inputs that reliably predicts the outcome. Several model families compete; the **validation** partition is used to tune and compare them (as Enterprise Miner does).*

In [17]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import brier_score_loss, accuracy_score

pipe = lambda est: Pipeline([("modify", FunctionTransformer(engineer)), ("prep",
def scores(y, p):
    return {"log_loss": log_loss(y, p), "brier": brier_score_loss(y, p), "roc_au

fitted, val_pred, fit_secs, search_size = {}, {}, {}, {}
probe_rows = np.random.default_rng(RANDOM_STATE).choice(len(X_tr), 5000, replace
assert pipe(LogisticRegression(max_iter=500)).fit(X_tr.iloc[probe_rows], y_tr[pr
```

Baseline — zone average

In [18]:
```
# Three baselines, from naive to strong:
#  1) base rate - every shot gets the training FG% (the no-information log loss)
#  2) zone average - the league FG% of the shot's official court zone
#  3) zone x shot family - the same, split by shot family (uses the strongest in
```

```
zone_keys = ["SHOT_ZONE_BASIC", "SHOT_ZONE_AREA", "SHOT_ZONE_RANGE"]
base_rate = train[TARGET].mean()
def group_rate(d, keys, min_shots=30):
    key = lambda f: f[keys].astype(str).agg("|".join, axis=1)
    g = train.assign(k=key(train)).groupby("k")[TARGET].agg(["mean", "size"])
    g = g[g["size"] >= min_shots]["mean"]
    return key(d).map(g).to_numpy(dtype=float)
def zone_baseline(d):
    return np.nan_to_num(group_rate(d, zone_keys), nan=base_rate)
def zone_family_baseline(d):
    d = d.assign(family=d["ACTION_TYPE"].map(action_family))
    p = group_rate(d, zone_keys + ["family"]) if "family" in d else None
    return np.where(np.isnan(p), zone_baseline(d), p)
train = train.assign(family=train["ACTION_TYPE"].map(action_family))
val_pred["Baseline: base rate"] = np.full(len(valid), base_rate)
val_pred["Baseline: zone average"] = zone_baseline(valid)
val_pred["Baseline: zone x shot family"] = zone_family_baseline(valid)
for b in ["Baseline: base rate", "Baseline: zone average", "Baseline: zone x sho
    print(f"{b:<30}", {k: round(v, 4) for k, v in scores(y_va, val_pred[b]).item
```

```
Baseline: base rate            {'log_loss': 0.691, 'brier': 0.2489, 'roc_auc': 0.
5, 'accuracy': 0.5325}
Baseline: zone average         {'log_loss': 0.6573, 'brier': 0.2324, 'roc_auc':
0.6339, 'accuracy': 0.6274}
Baseline: zone x shot family   {'log_loss': 0.6454, 'brier': 0.2277, 'roc_auc':
0.647, 'accuracy': 0.6289}
```

Regression node (logistic regression)

In [19]:

```
# Regression node, with the penalty strength C tuned on the validation partition
reg_search = []
for C in [0.01, 0.1, 1.0, 10.0]:
    t = time.time(); m = pipe(LogisticRegression(C=C, max_iter=2000)).fit(X_tr,
    reg_search.append({"C": C, "val_log_loss": log_loss(y_va, m.predict_proba(X_
reg_search = pd.DataFrame(reg_search); best_reg = reg_search.loc[reg_search["val
fitted["Regression"], fit_secs["Regression"], search_size["Regression"] = best_r
val_pred["Regression"] = fitted["Regression"].predict_proba(X_va)[:, 1]
display(reg_search.drop(columns="model").round(4)); print("Chosen C:", best_reg[
coef = pd.Series(fitted["Regression"].named_steps["model"].coef_[0], index=fitte
display(coef.sort_values().iloc[np.r_[0:6, -6:0]].round(3).to_frame("coefficient
```

	C	val_log_loss	seconds
0	0.01	0.6361	1.0972
1	0.10	0.6358	1.2583
2	1.00	0.6358	1.3386
3	10.00	0.6358	1.2885

```
Chosen C: 0.1
```

	coefficient (log-odds)
act__action_Tip Layup Shot	-0.901
act__action_Layup Shot	-0.555
act__action_Driving Layup Shot	-0.539
cat__action_family_tip/putback	-0.522
num__log_dist	-0.449
cat__action_family_alley-oop	-0.396
act__action_Putback Layup Shot	0.379
act__action_Alley Oop Dunk Shot	0.412
cat__distance_bin_0-1 ft	0.529
act__action_Cutting Dunk Shot	0.741
act__action_Running Dunk Shot	0.804
cat__action_family_dunk	0.812

Decision Tree node (depth chosen on the validation partition)

In [20]:
```
tree_search = []
for depth in [4, 6, 8, 10, 12, 15]:
    t = time.time()
    m = pipe(DecisionTreeClassifier(max_depth=depth, min_samples_leaf=100, rando
    tree_search.append({"max_depth": depth, "val_log_loss": log_loss(y_va, m.pre
tree_search = pd.DataFrame(tree_search)
best = tree_search.loc[tree_search["val_log_loss"].idxmin()]
fitted["Decision Tree"], fit_secs["Decision Tree"], search_size["Decision Tree"]
val_pred["Decision Tree"] = fitted["Decision Tree"].predict_proba(X_va)[:, 1]
display(tree_search.drop(columns="model").round(4)); print("Chosen depth (lowest
```

	max_depth	val_log_loss	seconds
0	4	0.6391	1.1816
1	6	0.6374	1.3151
2	8	0.6394	1.3638
3	10	0.6410	1.4937
4	12	0.6424	1.5544
5	15	0.6441	1.6283

```
Chosen depth (lowest validation log loss): 6
```

Neural Network node

In [21]:
```
nn_search = []
for layers, alpha, lr in [((32, 16), 1e-3, 1e-3), ((64, 32), 1e-4, 1e-3), ((16,)
    t = time.time()
    m = pipe(MLPClassifier(hidden_layer_sizes=layers, alpha=alpha, learning_rate
                           random_state=RANDOM_STATE)).fit(X_tr, y_tr)
    nn_search.append({"hidden_layers": str(layers), "alpha": alpha, "learning_ra
                      "val_log_loss": log_loss(y_va, m.predict_proba(X_va)[:, 1]
nn_search = pd.DataFrame(nn_search); best_nn = nn_search.loc[nn_search["val_log_
fitted["Neural Network"], fit_secs["Neural Network"], search_size["Neural Networ
val_pred["Neural Network"] = fitted["Neural Network"].predict_proba(X_va)[:, 1]
display(nn_search.drop(columns="model").round(4))
```

	hidden_layers	alpha	learning_rate	epochs	val_log_loss	seconds
0	(32, 16)	0.0010	0.0010	33	0.6355	9.4176
1	(64, 32)	0.0001	0.0010	18	0.6351	11.2862
2	(16,)	0.0100	0.0030	15	0.6354	3.9333
3	(64, 32)	0.0100	0.0003	14	0.6354	12.7310

Gradient Boosting node

In [22]:
```
gb_search = []
for lr, leaves in [(0.05, 15), (0.05, 31), (0.08, 31), (0.10, 63)]:
    t = time.time()
    m = pipe(HistGradientBoostingClassifier(learning_rate=lr, max_iter=500, max_
                                            early_stopping=True, random_state=RA
    gb_search.append({"learning_rate": lr, "max_leaf_nodes": leaves, "iterations
                      "val_log_loss": log_loss(y_va, m.predict_proba(X_va)[:, 1]
gb_search = pd.DataFrame(gb_search)
best_gb = gb_search.loc[gb_search["val_log_loss"].idxmin()]
fitted["Gradient Boosting"], fit_secs["Gradient Boosting"], search_size["Gradien
val_pred["Gradient Boosting"] = fitted["Gradient Boosting"].predict_proba(X_va)[
gb_search.drop(columns="model").round(4).to_csv(REP / "gradient_boosting_tuning.
display(gb_search.drop(columns="model").round(4))
print(f"Chosen: learning_rate={best_gb['learning_rate']}, max_leaf_nodes={best_g
      "(note: early stopping holds back 10 % of the training partition internall
```

	learning_rate	max_leaf_nodes	iterations	val_log_loss	seconds
0	0.05	15	157	0.6341	2.5608
1	0.05	31	145	0.6340	32.6413
2	0.08	31	60	0.6338	27.7926
3	0.10	63	47	0.6342	4.4230

```
Chosen: learning_rate=0.08, max_leaf_nodes=31 (note: early stopping holds back 10
% of the training partition internally)
```

Ensemble node

In [23]:
```
fitted["Ensemble"] = AveragedModel([fitted[m] for m in ["Regression", "Neural Ne
fit_secs["Ensemble"] = sum(fit_secs[m] for m in ["Regression", "Neural Network",
```

```
val_pred["Ensemble"] = fitted["Ensemble"].predict_proba(X_va)[:, 1]
print("Ensemble = average of Regression, Neural Network and Gradient Boosting pr
      "(the single tree is left out: it is the weakest learner and is dominated

# Ablations (not candidates for champion): the chosen boosting configuration wit
# which is recorded after the play and could partly encode the outcome, and (b)
gb_params = dict(learning_rate=best_gb["learning_rate"], max_iter=500, max_leaf_
                 l2_regularization=1.0, early_stopping=True, random_state=RANDOM
ablations = {"Ablation: boosting, location + clock only (no shot type)": make_pr
             "Ablation: boosting without the shot angle": make_prep(numeric=[c f
for name, prep in ablations.items():
    m = Pipeline([("modify", FunctionTransformer(engineer)), ("prep", prep), ("m
    val_pred[name] = m.predict_proba(X_va)[:, 1]
    print(f"{name}: validation log loss {log_loss(y_va, val_pred[name]):.4f}")
probe_p = fitted["Gradient Boosting"].predict_proba(probe)[:, 1]
print("Probe xFG from the boosting model:", dict(zip(probe.index, probe_p.round(
```

```
Ensemble = average of Regression, Neural Network and Gradient Boosting probabilit
ies (the single tree is left out: it is the weakest learner and is dominated by b
oosting, which is itself built from trees)
Ablation: boosting, location + clock only (no shot type): validation log loss 0.6
471
Ablation: boosting without the shot angle: validation log loss 0.6340
Probe xFG from the boosting model: {'missing location, clock and shot type': np.f
loat64(0.551), 'unseen label, still a dunk at the rim': np.float64(0.924)}
```

Model families in the competition

Node	What it is	Assumptions / strengths	Settings tried on validation → chosen
Base rate	every shot gets the training FG%	the no-information benchmark (log loss 0.691)	—
Zone average	league FG% of the official zone	no learning beyond the zone; the "analyst's" benchmark	—
Zone × shot family	league FG% of the zone **and** shot family	a strong benchmark that uses the same key input as the models	—
Regression	logistic regression on scaled inputs + one-hot shot type, shot family and distance bins	linear in log-odds; the bins supply the non-linearity. Overlapping encodings are kept apart only by the L2 penalty, so coefficients show direction (dunk family +0.81, tip layups −0.90, 0–1 ft +0.53 log-odds) rather than clean effects	4 values of C → C = 0.1
Decision Tree	single CART tree	captures interactions, easy to read; high variance	6 depths → depth 6
Neural Network	MLP	flexible smooth functions; needs scaled inputs	4 architectures / penalties / learning rates → 16 units, α = 0.01
Gradient Boosting	histogram gradient-boosted trees	strongest on tabular data; interactions + non-linearity. Early	4 settings → learning rate 0.08,

Node	What it is	Assumptions / strengths	Settings tried on validation → chosen
		stopping holds back 10 % of training internally	31 leaves, 60 iterations
Ensemble	average of regression, NN and boosting probabilities	reduces variance if the members disagree; the tree is left out as the weakest learner	—

🔍 Critic review — Model

Each phase was reviewed in three rounds with the expert-critic prompt below. Round 1 was run in the drafting session. Rounds 2 and 3 were run by fresh, independent Claude sessions that saw only the executed notebook, its figures and the app files. Full reviews and point-by-point responses: `critiques/round_1.md`, `round_2.md`, `round_3.md`.

> *You are a world-renowned authority on the SEMMA data mining methodology and SAS Enterprise Miner, a sought-after keynote speaker, and the author of several award-winning books on SEMMA and predictive modeling. Review the **Model** phase of my NBA expected-field-goal-percentage project below. Check it against everything SEMMA's Model step should accomplish (and the Enterprise Miner nodes that implement it). Focus especially on: whether the competing model families are well chosen and fairly configured, whether tuning used only the validation partition, whether a meaningful baseline is included, and whether settings are documented. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Unfair comparison — the tree was tuned on validation but boosting was not; neural network untuned.	Tuned boosting over 4 settings on validation (lr 0.05, 31 leaves); NN left as is (ensemble including it does not beat boosting).
2	Unequal tuning; champion picked on a 0.0006 gap without uncertainty; weak baseline.	Every family tuned; game-level paired bootstrap; base-rate and zone × shot-family baselines.
3	No model without the post-hoc shot-type label.	Ablations: without the shot type 0.6471 (worse than the zone × family baseline); without the angle no measurable change.

A — Assess

Goal: evaluate the usefulness and reliability of the findings — compare models, check calibration, confirm on untouched data, and score new data.

A.1 Model Comparison (validation partition)

In [24]:
```
comparison = pd.DataFrame({m: scores(y_va, p) for m, p in val_pred.items()}).T
comparison["settings_tried"] = pd.Series(search_size)
comparison["fit_seconds"] = pd.Series(fit_secs).round(1)
comparison = comparison.sort_values("log_loss").round(4)
comparison.to_csv(REP / "model_comparison_validation.csv"); display(comparison)
CHAMPION = comparison.drop(index=[i for i in comparison.index if i.startswith(("

# Are the differences bigger than noise? Paired bootstrap over GAMES of the vali
# are not independent): 95 % interval of each model's log-loss difference agains
games = valid["GAME_ID"].to_numpy(); ug, ginv = np.unique(games, return_inverse=
shot_ll = lambda p: -(y_va * np.log(np.clip(p, 1e-12, 1)) + (1 - y_va) * np.log(
game_sum = lambda v: np.bincount(ginv, weights=v, minlength=len(ug))
n_shots = np.bincount(ginv, minlength=len(ug)); champ_ll = game_sum(shot_ll(val_
rng = np.random.default_rng(RANDOM_STATE); boot_idx = rng.integers(0, len(ug), (
paired = []
for m, p in val_pred.items():
    if m == CHAMPION: continue
    d = game_sum(shot_ll(p)) - champ_ll
    diffs = d[boot_idx].sum(1) / n_shots[boot_idx].sum(1)
    paired.append({"model": m, "log_loss_minus_champion": d.sum() / n_shots.sum(
                   "ci_95_low": np.quantile(diffs, .025), "ci_95_high": np.quant
paired = pd.DataFrame(paired).set_index("model").sort_values("log_loss_minus_cha
paired["clearly_worse"] = paired["ci_95_low"] > 0
paired.to_csv(REP / "model_comparison_bootstrap.csv"); display(paired)
print(f"Champion (lowest validation log loss): {CHAMPION} | models not clearly w
```

	log_loss	brier	roc_auc	accuracy	settings_tried	fit_seconds
Gradient Boosting	0.6338	0.2227	0.6659	0.6370	4.0	27.8
Ensemble	0.6340	0.2227	0.6652	0.6362	1.0	40.3
Ablation: boosting without the shot angle	0.6340	0.2228	0.6650	0.6359	NaN	NaN
Neural Network	0.6351	0.2233	0.6619	0.6342	4.0	11.3
Regression	0.6358	0.2236	0.6618	0.6349	4.0	1.3
Decision Tree	0.6374	0.2239	0.6593	0.6340	6.0	1.3
Baseline: zone x shot family	0.6454	0.2277	0.6470	0.6289	NaN	NaN
Ablation: boosting, location + clock only (no shot type)	0.6471	0.2280	0.6507	0.6287	NaN	NaN
Baseline: zone average	0.6573	0.2324	0.6339	0.6274	NaN	NaN
Baseline: base rate	0.6910	0.2489	0.5000	0.5325	NaN	NaN

model	log_loss_minus_champion	ci_95_low	ci_95_high	clearly_worse
Ensemble	0.00013	-0.00027	0.00052	False
Ablation: boosting without the shot angle	0.00015	-0.00014	0.00042	False
Neural Network	0.00130	0.00060	0.00202	True
Regression	0.00195	0.00120	0.00270	True
Decision Tree	0.00333	0.00209	0.00495	True
Baseline: zone x shot family	0.01160	0.01000	0.01310	True
Ablation: boosting, location + clock only (no shot type)	0.01323	0.01180	0.01471	True
Baseline: zone average	0.02343	0.02144	0.02527	True
Baseline: base rate	0.05720	0.05425	0.06003	True

```
Champion (lowest validation log loss): Gradient Boosting | models not clearly wor
se: ['Ensemble', 'Ablation: boosting without the shot angle']
```

Why log loss? xFG% is used as a **probability** (expected points = xFG% × shot value), so the selection criterion must reward accurate probabilities — not just ranking (AUC) or a 50 % cutoff (accuracy). Brier score agrees with log loss on the ranking of every model.

Result. Gradient boosting has the lowest validation log loss (0.6338). A paired bootstrap over the 1,166 validation games shows it is **statistically tied with the ensemble** (difference +0.0001, 95 % CI −0.0003 to +0.0005). Boosting is kept as champion because it is a single, simpler model. The neural network (+0.0012), regression (+0.0020) and the tree (+0.0033) are clearly worse.

Every learned model beats all three baselines: the zone average (0.6573), the stronger zone × shot-family average (0.6454) and the base rate (0.6910). The champion's honest gain over the strong baseline is 0.0116 log loss (1.8 %, CI 0.0100–0.0131), and AUC rises from 0.647 to 0.666. The gain looks modest because most of a shot's outcome is genuinely random once you know only *where* and *what* it was; the model cannot see the defender, the shooter's balance or the pass.

The champion relies on vertical location (shuffling it raises log loss by 0.043), shot type (0.039) and horizontal location (0.037); the game clock adds about 0.001.

Ablations (not candidates for champion):

- **Without the shot-type label** (location + clock only), the same boosting configuration scores **0.6471**. That is clearly worse than the champion (+0.0132) and even worse than the zone × shot-family baseline. So **most of the model's edge comes from the scorer's shot-type label**. That label is recorded after the play: if

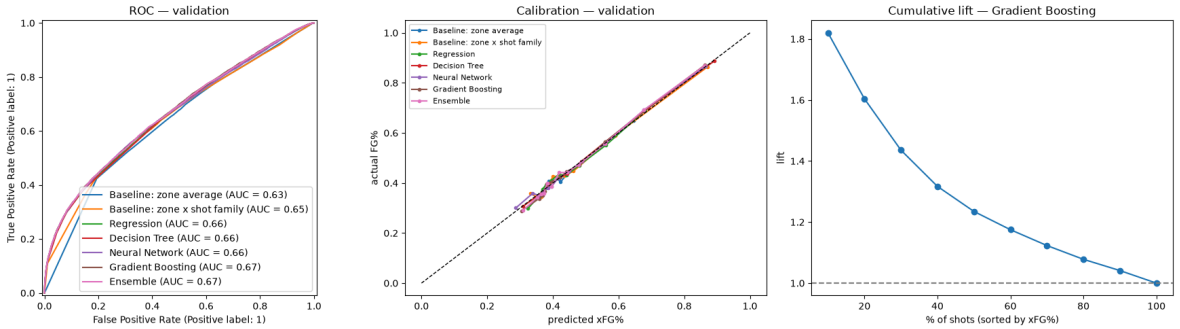
scorers sometimes log a missed dunk as a layup, part of this gain is leakage. The location-only model's gain over the zone average (0.0102) is free of that risk.

- **Without the shot angle**, it scores 0.6340: no measurable difference (CI −0.0001 to +0.0004). The angle is kept because it is harmless for trees, but it adds nothing beyond distance and the corner flag. Its top-rated 10 % of shots go in about 1.8 times as often as average.

In [25]:
```
from sklearn.metrics import RocCurveDisplay
from sklearn.calibration import calibration_curve
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for m, p in val_pred.items():
    if m == "Baseline: base rate" or m.startswith("Ablation"): continue
    RocCurveDisplay.from_predictions(y_va, p, ax=axes[0], name=m)
    frac_pos, mean_pred = calibration_curve(y_va, p, n_bins=12, strategy="quanti
    axes[1].plot(mean_pred, frac_pos, marker="o", ms=3, label=m)
axes[0].set_title("ROC — validation"); axes[1].plot([0, 1], [0, 1], "k--", lw=1)
axes[1].set(title="Calibration — validation", xlabel="predicted xFG%", ylabel="a

# Cumulative lift chart (Enterprise Miner's standard assessment plot)
order = np.argsort(-val_pred[CHAMPION]); base = y_va.mean()
deciles = np.array_split(y_va[order], 10)
cum_lift = np.cumsum([d.sum() for d in deciles]) / np.cumsum([len(d) for d in de
axes[2].plot(range(10, 101, 10), cum_lift, marker="o"); axes[2].axhline(1, c="gr
axes[2].set(title=f"Cumulative lift — {CHAMPION}", xlabel="% of shots (sorted by
savefig("assess_model_comparison")

# What does the champion rely on? Permutation importance of the six raw inputs (
from sklearn.inspection import permutation_importance
sub = X_va.sample(10_000, random_state=RANDOM_STATE)
pi = permutation_importance(fitted[CHAMPION], sub, y_va[X_va.index.get_indexer(s
                            n_repeats=3, random_state=RANDOM_STATE)
display(pd.Series(pi.importances_mean, index=RAW_INPUTS).sort_values(ascending=F
```



	increase in log loss when shuffled
LOC_Y	0.0426
ACTION_TYPE	0.0385
LOC_X	0.0372
MINUTES_REMAINING	0.0010
SECONDS_REMAINING	0.0006
PERIOD	0.0002

A.2 Cutoff (how predictions translate into make/miss calls)

In [26]:
```
p = val_pred[CHAMPION]
cutoff = pd.DataFrame([{"cutoff": c, "predicted_makes_%": (p >= c).mean() * 100,
                        "precision": y_va[p >= c].mean() if (p >= c).any() else
                        "recall": (y_va[p >= c].sum() / y_va.sum())} for c in [0
cutoff.to_csv(REP / "assess_cutoff.csv", index=False); display(cutoff)
```

	cutoff	predicted_makes_%	precision	recall
0	0.3	97.237	0.474	0.987
1	0.4	53.180	0.567	0.645
2	0.5	26.952	0.694	0.400
3	0.6	17.839	0.768	0.293
4	0.7	10.972	0.839	0.197

xFG% is used as a probability, so no single cutoff is required. The table shows what labels would mean: shots rated ≥ 60 % go in 77 % of the time (17.8 % of shots), and ≥ 70 % go in 84 % of the time — "high-quality shot" labels are reliable.

A.3 Final check on untouched data — test partition and the unseen 2024-25 season

In [27]:
```
oot = load_season(OUT_OF_TIME_SEASON)
champ = fitted[CHAMPION]
final = {
    "train (2019-24 sample)": (y_tr, champ.predict_proba(X_tr)[:, 1], train),
    "validate (2019-24 sample)": (y_va, champ.predict_proba(X_va)[:, 1], valid),
    "test (2019-24 sample)": (y_te, champ.predict_proba(X_te)[:, 1], test),
    f"out-of-time ({oot['SEASON'].iloc[0]}, all {len(oot):,} shots)": (oot[TARGE
}
final_table = pd.DataFrame({k: {**scores(y, p), "zone_baseline_log_loss": log_lo
                                "zone_x_family_baseline_log_loss": log_loss(y, z
                                "mean_xFG": p.mean(), "actual_FG": y.mean()}
                            for k, (y, p, d) in final.items()}).T.round(4)
final_table.to_csv(REP / "assess_final_performance.csv"); display(final_table)
```

	log_loss	brier	roc_auc	accuracy	zone_baseline_log_loss	zone_x_family_baselin
train (2019-24 sample)	0.6320	0.2220	0.6705	0.6356	0.6595	
validate (2019-24 sample)	0.6338	0.2227	0.6659	0.6370	0.6573	
test (2019-24 sample)	0.6339	0.2228	0.6640	0.6348	0.6584	
out-of-time (2024-25, all 219,527 shots)	0.6370	0.2241	0.6588	0.6303	0.6577	

In [28]:
```
# Calibration on the unseen 2024-25 season: by decile, and by segment (zone, 2PT
# Player rankings depend on calibration WITHIN shot types, which a decile table
y_oot, p_oot = final[list(final)[-1]][:2]
cal = pd.DataFrame({"xFG": p_oot, "made": y_oot})
cal["decile"] = pd.qcut(cal["xFG"], 10, labels=[f"D{i}" for i in range(1, 11)])
cal = cal.groupby("decile", observed=True).agg(shots=("made", "size"), mean_xFG=
cal["gap_pct_points"] = (cal["actual_FG"] - cal["mean_xFG"]) * 100
cal = cal.round(3); cal.to_csv(REP / "calibration_by_decile_2024_25.csv"); displ
print(f"Largest decile gap: {cal['gap_pct_points'].abs().max():.1f} percentage p

seg_cal = []
oot_seg = oot.assign(xFG=p_oot, family=oot["ACTION_TYPE"].map(action_family))
for col in ["SHOT_ZONE_BASIC", "SHOT_TYPE", "family"]:
    g = oot_seg.groupby(col, observed=True).agg(shots=(TARGET, "size"), mean_xFG
    g = g[g["shots"] >= 500]
    seg_cal.append(g.assign(segment_by=col, gap_pct_points=(g["actual_FG"] - g["
seg_cal = pd.concat(seg_cal)[["segment_by", "segment", "shots", "mean_xFG", "act
seg_cal.to_csv(REP / "calibration_by_segment_2024_25.csv", index=False); display
print(f"Largest segment gap (segments with 500+ shots): {seg_cal['gap_pct_points
```

decile	shots	mean_xFG	actual_FG	gap_pct_points
D1	21985	0.310	0.309	-0.067
D2	21945	0.361	0.353	-0.739
D3	22077	0.371	0.366	-0.500
D4	22007	0.378	0.376	-0.191
D5	21785	0.393	0.395	0.168
D6	21917	0.421	0.428	0.653
D7	21954	0.444	0.459	1.534
D8	21955	0.503	0.507	0.376
D9	21951	0.627	0.630	0.257
D10	21951	0.839	0.850	1.148

```
Largest decile gap: 1.5 percentage points
```

	segment_by	segment	shots	mean_xFG	actual_FG	gap_pct_points
0	SHOT_ZONE_BASIC	Above the Break 3	68359	0.362	0.353	-0.921
1	SHOT_ZONE_BASIC	Backcourt	517	0.072	0.023	-4.893
2	SHOT_ZONE_BASIC	In The Paint (Non-RA)	44475	0.432	0.444	1.133
3	SHOT_ZONE_BASIC	Left Corner 3	12188	0.383	0.386	0.323
4	SHOT_ZONE_BASIC	Mid-Range	21401	0.421	0.417	-0.351
5	SHOT_ZONE_BASIC	Restricted Area	61190	0.653	0.664	1.100
6	SHOT_ZONE_BASIC	Right Corner 3	11397	0.382	0.390	0.802
0	SHOT_TYPE	2PT Field Goal	127072	0.536	0.545	0.865
1	SHOT_TYPE	3PT Field Goal	92455	0.366	0.360	-0.564
0	family	alley-oop	1001	0.636	0.647	1.104
1	family	bank shot	992	0.521	0.510	-1.103
2	family	dunk	13379	0.874	0.882	0.779
3	family	fadeaway/turnaround	9439	0.422	0.415	-0.755
4	family	floater	20798	0.453	0.445	-0.825
5	family	hook	6017	0.480	0.481	0.137
6	family	jump shot	69886	0.371	0.375	0.370
7	family	layup	48253	0.545	0.565	2.056
8	family	pull-up/step-back	41206	0.386	0.374	-1.173
9	family	tip/putback	8556	0.552	0.545	-0.694

```
Largest segment gap (segments with 500+ shots): 4.9 percentage points
```

No over-fitting, and it travels to a new season.

- **Over-fitting.** Training log loss (0.6320) is only slightly below validation (0.6338) and test (0.6339), so the model does not memorise its training shots. Test equals validation, so choosing the champion on validation did not over-fit either.
- **New season.** On the full 2024-25 season, which the model never saw, log loss is 0.6370 against 0.6466 for the zone × family baseline and 0.6577 for the zone baseline, and AUC is 0.659: a small, expected decline.
- **Calibration in the large.** Predicted 46.5 % vs actual 46.7 %.
- **Calibration by decile.** Every gap is ≤ 1.5 percentage points (largest in D7: 44.4 % predicted, 45.9 % actual).
- **Calibration by segment.** Every shot zone and shot family with 500+ shots is within 2.1 points, with one exception: backcourt heaves (517 shots) are over-predicted, 7.2 % vs 2.3 %. Two systematic gaps matter for players: **layups go in 2.1 points more than predicted** and pull-ups 1.2 points less. Players who live at the rim are therefore slightly flattered by the leaderboard in A.4, and pull-up shooters slightly penalised.

The three success criteria are met.

Process note: the test partition and the 2024-25 season were never used to fit or choose anything. But their results were visible in every review round while the analysis was revised (rim bins, shot families, extra baselines). "Unseen" therefore holds for training, not for model development. The effect is likely small, because test still equals validation.

A.4 Score & Segment Profile — shot-making over expected (2024-25)

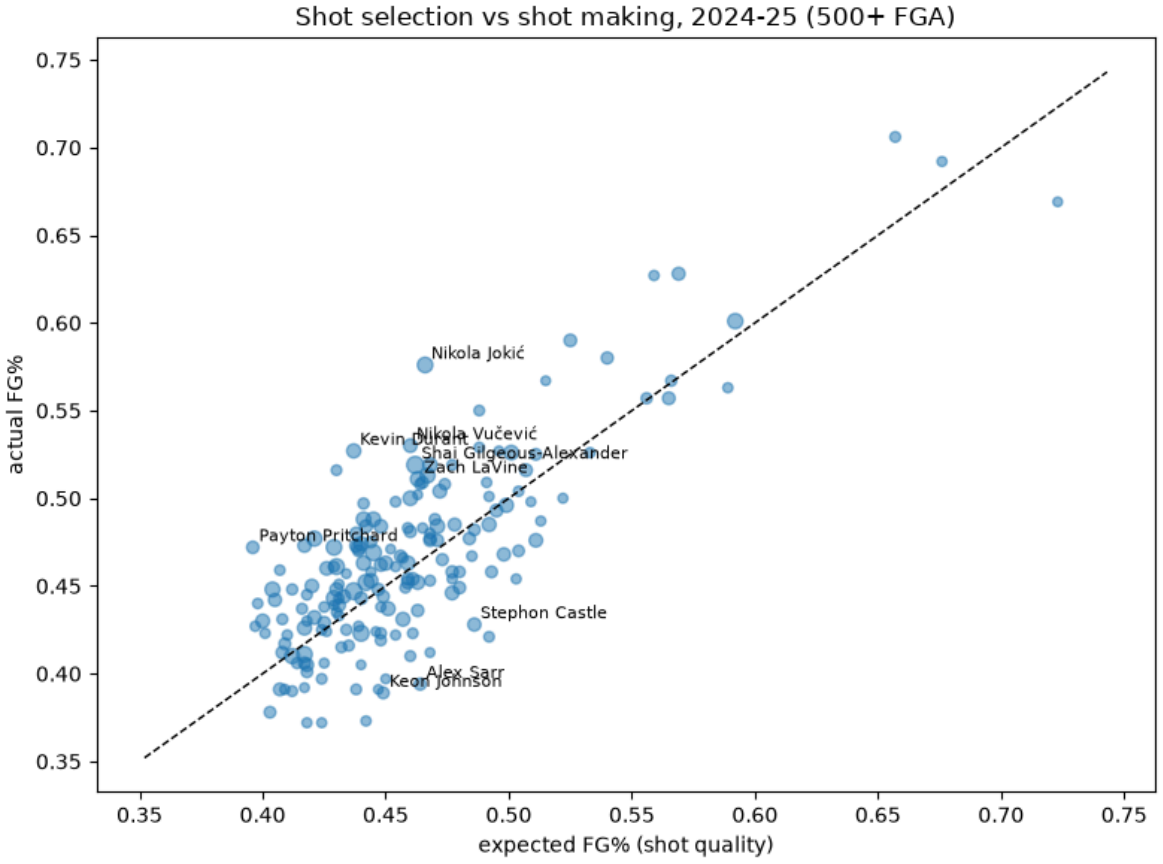
In [29]:
```
# Shot making over expected, 2024-25. Players are grouped by PLAYER_ID (names ar
# different seasons). Points come from the OFFICIAL shot type. Each player gets
# his own shots) and an empirical-Bayes estimate shrunk towards the group mean (
oot = oot.assign(xFG=p_oot, pts=np.where(oot["SHOT_TYPE"] == "3PT Field Goal", 3
players = oot.groupby("PLAYER_ID").agg(PLAYER_NAME=("PLAYER_NAME", lambda s: str
                                       FGA=(TARGET, "size"), FG_pct=(TARGET, "me
                                       var_sum=("xFG", lambda s: (s * (1 - s)).s
                                       points=(TARGET, lambda s: (s * oot.loc[s.
                                       expected_points=("xFG", lambda s: (s * oo
players = players[players["FGA"] >= 500].assign(FG_over_expected=lambda d: d["FG
                                                points_over_expected=lambda d: d
                                                se=lambda d: np.sqrt(d["var_sum"
mu = players["FG_over_expected"].mean()
true_var = max(players["FG_over_expected"].var() - (players["se"] ** 2).mean(),
players["FGOE_shrunk"] = mu + (players["FG_over_expected"] - mu) * true_var / (t
players["ci_95_low"], players["ci_95_high"] = players["FG_over_expected"] - 1.96
players["clearly_above_or_below_0"] = (players["ci_95_low"] > 0) | (players["ci_
players = players.drop(columns="var_sum").reset_index().set_index("PLAYER_NAME")
players.to_csv(REP / "shot_making_leaderboard_2024_25.csv")
print(f"{len(players)} players with 500+ FGA in {oot['SEASON'].iloc[0]}; group m
      f"typical 95 % interval ±{1.96 * players['se'].median() * 100:.1f} pts; {p
      f"different from 0 (about {0.05 * len(players):.0f} would be by chance)")
display(players.head(15)); display(players.tail(5))
```

181 players with 500+ FGA in 2024-25; group mean FG over expected +0.7 pts; typical 95 % interval ±3.3 pts; 62 players clearly different from 0 (about 9 would be by chance)

PLAYER_NAME	PLAYER_ID	team	FGA	FG_pct	xFG_pct	points	expected_points
Nikola Jokić	203999	Denver Nuggets	1364	0.576	0.466	1710	1383.216
Kevin Durant	201142	Phoenix Suns	1124	0.527	0.437	1344	1118.203
Shai Gilgeous-Alexander	1628983	Oklahoma City Thunder	1656	0.519	0.462	1883	1686.676
Payton Pritchard	1630202	Boston Celtics	866	0.472	0.396	1073	908.530
Zach LaVine	203897	Chicago Bulls	1223	0.511	0.463	1489	1328.734
Nikola Vučević	202696	Chicago Bulls	1038	0.530	0.460	1229	1072.905
DeMar DeRozan	201942	Sacramento Kings	1308	0.477	0.421	1332	1192.267
Stephen Curry	201939	Golden State Warriors	1258	0.448	0.404	1439	1299.791
Tyrese Haliburton	1630169	Indiana Pacers	1006	0.473	0.417	1170	1040.709
Ty Jerome	1629660	Cleveland Cavaliers	618	0.516	0.430	749	621.320
Tyler Herro	1629639	Miami Heat	1378	0.472	0.429	1553	1425.381
Pascal Siakam	1627783	Indiana Pacers	1182	0.519	0.468	1352	1224.540
Domantas Sabonis	1627734	Sacramento Kings	893	0.590	0.525	1119	994.114
LeBron James	2544	Los Angeles Lakers	1270	0.513	0.467	1451	1329.593
Jaren Jackson Jr.	1628991	Memphis Grizzlies	1213	0.488	0.441	1330	1212.467

PLAYER_NAME	PLAYER_ID	team	FGA	FG_pct	xFG_pct	points	expected_points
Scottie Barnes	1630567	Toronto Raptors	1063	0.446	0.477	1024	1115.272
Bilal Coulibaly	1641731	Washington Wizards	624	0.421	0.492	589	697.822
Keon Johnson	1630553	Brooklyn Nets	779	0.389	0.449	732	845.978
Alex Sarr	1642259	Washington Wizards	828	0.394	0.464	757	893.651
Stephon Castle	1642264	San Antonio Spurs	988	0.428	0.486	941	1079.439

In [30]:
```
fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(players["xFG_pct"], players["FG_pct"], s=players["FGA"] / 25, alpha=.
lim = [players[["xFG_pct", "FG_pct"]].min().min() - .02, players[["xFG_pct", "FG
ax.plot(lim, lim, "k--", lw=1)
for name, r in pd.concat([players.head(6), players.tail(3)]).iterrows():
    ax.annotate(name, (r["xFG_pct"], r["FG_pct"]), fontsize=8, xytext=(3, 3), te
ax.set(xlabel="expected FG% (shot quality)", ylabel="actual FG%", title="Shot se
savefig("assess_shot_making")
```

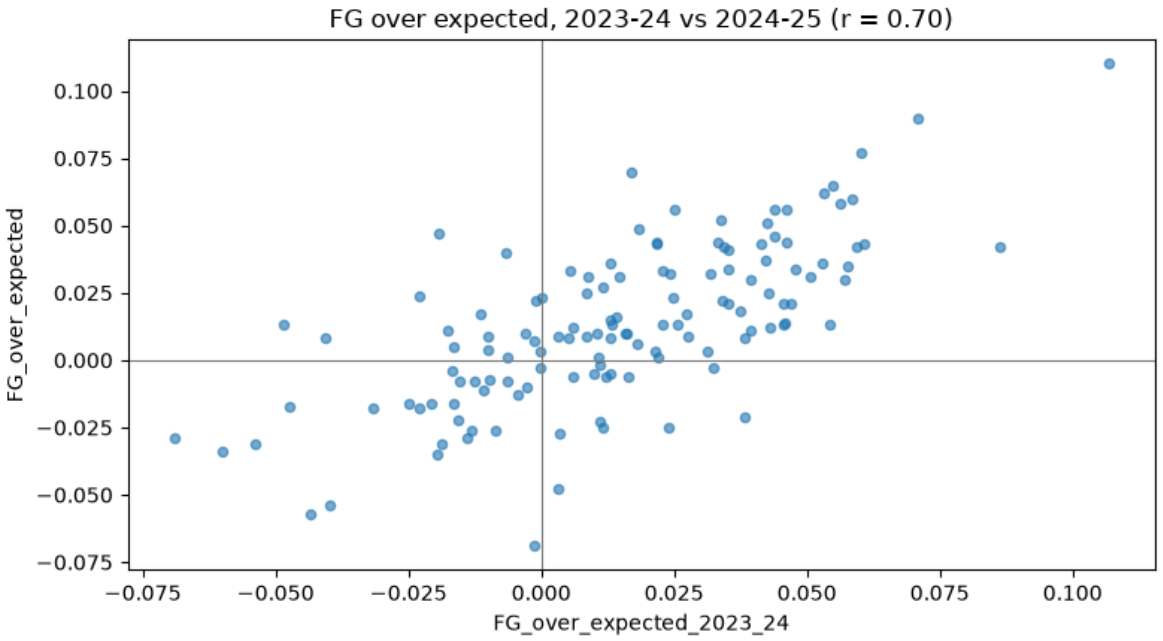


In [31]:
```
# Is "FG over expected" persistent? Compare 2023-24 with 2024-25 for the same PL
# FG% and the xFG% of a player's shots persist on their own. Persistence alone c
# shot context the model does not see (defender distance, self-created vs assist
prev = population[population["SEASON"] == "2023-24"]
```

```
prev = prev.assign(xFG=champ.predict_proba(prev[RAW_INPUTS])[:, 1])
prev_players = prev.groupby("PLAYER_ID").agg(FGA=(TARGET, "size"), FG_pct=(TARGE
prev_players = prev_players[prev_players["FGA"] >= 500].assign(FG_over_expected=
both = players.reset_index().set_index("PLAYER_ID")[["PLAYER_NAME", "FG_over_exp
    prev_players[["FG_over_expected", "FG_pct", "xFG_pct"]], how="inner", rsuffi
def r_ci(a, b):
    r = np.corrcoef(a, b)[0, 1]; z, se = np.arctanh(r), 1 / np.sqrt(len(a) - 3)
    return r, np.tanh(z - 1.96 * se), np.tanh(z + 1.96 * se)
persistence = pd.DataFrame([dict(zip(["r", "ci_95_low", "ci_95_high"], r_ci(both
                           for m in ["FG_over_expected", "FG_pct", "xFG_pct"]])
both.to_csv(REP / "shot_making_persistence.csv"); persistence.to_csv(REP / "shot
r = persistence.loc["FG_over_expected", "r"]
print(f"{len(both)} players with 500+ FGA in both seasons (only players who kept
display(persistence)
fig, ax = plt.subplots()
both.plot.scatter("FG_over_expected_2023_24", "FG_over_expected", alpha=.6, ax=a
ax.axhline(0, c="gray", lw=.8); ax.axvline(0, c="gray", lw=.8); savefig("assess_
```

```
131 players with 500+ FGA in both seasons (only players who kept a big role in bo
th seasons qualify):
```

measure	r	ci_95_low	ci_95_high
FG_over_expected	0.70	0.60	0.78
FG_pct	0.86	0.81	0.90
xFG_pct	0.94	0.92	0.96



Reading the chart: above the diagonal = the player makes more than his shots would predict (**shot making**); further right = he takes higher-quality shots (**shot selection**). With about 1,000 attempts, a player's FG over expected has a 95 % interval of about **±3.3 points**. 62 of the 181 players are clearly different from zero (about 9 would be by chance alone), and the rest are within noise. The leaderboard also shows an empirical-Bayes **shrunk** estimate that pulls noisy players towards the group mean (+0.7 points: players with 500+ attempts are, as a group, better than average).

- **Nikola Jokić** leads by far: 57.6 % on shots worth 46.6 %, **+327 points over expected** (FG over expected +11.0 points, 95 % CI +8.5 to +13.6; shrunk +9.3). His shots are not unusually easy; he simply converts them.
- **Kevin Durant** (+226) takes clearly harder shots (xFG 43.7 %) and still makes 52.7 % of them. **Shai Gilgeous-Alexander** is +196.
- **Stephen Curry**'s xFG% is low (40.4 %) because of his shot diet (long threes), yet he is +139: raw FG% (44.8 %) badly undersells him.
- At the bottom: Stephon Castle (−138), Alex Sarr (−137), Keon Johnson (−114), Bilal Coulibaly (−109) and Scottie Barnes (−91). Three of the five are in their first or second NBA season.

Skill, or a persistent residual? For the 131 players with 500+ attempts in both 2023-24 and 2024-25 (matched by player ID; names are spelled with and without accents in different seasons):

- FG over expected correlates **r = 0.70** across seasons (95 % CI 0.60–0.78);
- raw FG% correlates more (r = 0.86), and the xFG% of a player's shots most of all (0.94).

Shot *diet* is very stable, and shot-making-over-expected persists moderately. It can only be called a **persistent residual**: real shot-making skill plus shot context the model cannot see but that is stable for a player's role (defender distance, self-created vs assisted, post-ups). The sample also keeps only players who held a big role in both seasons. And 2023-24 xFG is partly in-sample: about 12 % of those shots were training data, which slightly flatters that season.

What the model does not know: defender distance, catch-and-shoot vs off-the-dribble beyond the `ACTION_TYPE` label, fatigue, and play context. Player-tracking data would close much of that gap, but it is not public.

🔍 Critic review — Assess

Each phase was reviewed in three rounds with the expert-critic prompt below. Round 1 was run in the drafting session. Rounds 2 and 3 were run by fresh, independent Claude sessions that saw only the executed notebook, its figures and the app files. Full reviews and point-by-point responses: `critiques/round_1.md`, `round_2.md`, `round_3.md`.

> *You are a world-renowned authority on the SEMMA data mining methodology and SAS Enterprise Miner, a sought-after keynote speaker, and the author of several award-winning books on SEMMA and predictive modeling. Review the **Assess** phase of my NBA expected-field-goal-percentage project below. Check it against everything SEMMA's Assess step should accomplish (and the Enterprise Miner nodes that implement it). Focus especially on: whether the champion was selected on an appropriate criterion, whether the test partition and out-of-time season were used only once, whether calibration was checked, and whether the player-level findings are interpreted with appropriate caveats. List (1) anything missing, (2)*

> *anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Calibration only checked on average; the leaderboard could be noise; explain the modest AUC gain.	Added decile calibration on 2024-25 (all gaps ≤ 0.9 pts); added season-to-season persistence check (r = 0.68); explained irreducible randomness and log-loss gain.
2	"Repeatable skill" overclaimed; only decile calibration; no player uncertainty; deployment unfinished; SAS export kept the target leak.	Persistence benchmarked; segment calibration; player CIs and shrinkage; refit for deployment; SAS export fixed.
3	Seasons joined on player names (accents differ) → Jokić and 6 others dropped; shrinkage toward 0; SAS partition step wrong.	Joined on PLAYER_ID (131 players, r = 0.70); shrinkage toward the group mean; three role-tagged SAS files; full disclosure of hold-out exposure.

Beyond SEMMA — Deployment

*SEMMA stops at Assess (scoring is part of it); the assignment requires delivery all the way to deployment. The champion configuration is **refit on train + validate** (160,078 shots, as the learning curve suggests more data helps a little). It is checked once on 2024-25 (log loss 0.6370, the same as the assessed model) and published as a small web app: pick a spot on the court and a shot type → expected FG% and expected points, plus the 2024-25 shot-making leaderboard with 95 % intervals.*

In [32]:
```
import joblib
from sklearn.base import clone
# Deployment: refit the CHAMPION CONFIGURATION on train + validate (twice the da
# learning curve in S.3 was still falling at 120k), then check it once on the ou
X_trva = pd.concat([X_tr, X_va]); y_trva = np.r_[y_tr, y_va]
deployed = clone(champ).fit(X_trva, y_trva) if hasattr(champ, "get_params") else
p_dep = deployed.predict_proba(oot[RAW_INPUTS])[:, 1]
print(f"Deployed model (trained on {len(X_trva):,} shots): out-of-time log loss
      f"(assessed model {log_loss(y_oot, p_oot):.4f}), mean xFG {p_dep.mean():.4
joblib.dump(deployed, APP / "model.joblib", compress=3)

counts = train["ACTION_TYPE"].value_counts()
actions = counts[counts >= 50].index.astype(str).tolist()                # shot t
support = engineer(train).groupby(["action_family", "distance_bin"]).size()
meta = {"champion": CHAMPION, "created": datetime.now().isoformat(timespec="seco
        "validation": comparison.loc[CHAMPION].to_dict(), "final": final_table.t
        "deployed_out_of_time_log_loss": round(float(log_loss(y_oot, p_dep)), 4)
        "actions": actions, "support": {f"{f}|{b}": int(n) for (f, b), n in supp
        "trained_on": f"{len(X_trva):,} shots (train + validate), seasons {POPUL
        "versions": {"python": platform.python_version(), "numpy": np.__version_
(APP / "model_metadata.json").write_text(json.dumps(meta, indent=2, default=str)
```

```
players.reset_index().to_csv(APP / "leaderboard.csv", index=False)
print(f"model.joblib: {(APP / 'model.joblib').stat().st_size / 1e6:.1f} MB; {len
```

```
Deployed model (trained on 160,078 shots): out-of-time log loss 0.6370 (assessed
model 0.6370), mean xFG 0.4644 vs actual 0.4672
model.joblib: 0.2 MB; 44 shot types offered in the app
```

In [33]:

```
APP_CODE = r"""
import json
from pathlib import Path
import joblib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import streamlit as st
from matplotlib.patches import Arc, Circle, Rectangle
from shot_features import engineer   # score code - must sit next to this file

HERE = Path(__file__).parent
st.set_page_config(page_title="NBA xFG%", page_icon="🏀")

@st.cache_resource
def load():
    return joblib.load(HERE / "model.joblib"), json.loads((HERE / "model_metadat

model, meta = load()
st.title("🏀 NBA expected field-goal %")
st.caption(f"{meta['champion']} · trained on {meta['trained_on']}")

def court(ax):
    for p in [Circle((0, 0), 7.5, fill=False), Rectangle((-80, -47.5), 160, 190,
              Arc((0, 142.5), 120, 120, theta1=0, theta2=180), Arc((0, 0), 80, 8
              Rectangle((-220, -47.5), 0, 140), Rectangle((220, -47.5), 0, 140),
              Rectangle((-250, -47.5), 500, 470, fill=False)]:
        p.set_edgecolor("#555"); ax.add_patch(p)
    ax.set_xlim(-250, 250); ax.set_ylim(-47.5, 422.5); ax.set_aspect("equal"); a

shot, board = st.tabs(["Evaluate a shot", "2024-25 shot-making leaderboard"])
with shot:
    left, right = st.columns([1, 1])
    with left:
        x_ft = st.slider("Feet left (-) / right (+) of the hoop", -25.0, 25.0, 0
        y_ft = st.slider("Feet out from the hoop", -4.0, 40.0, 15.0, 0.5)
        action = st.selectbox("Shot type", meta["actions"], index=0)
        period = st.selectbox("Period", [1, 2, 3, 4, 5], index=0, format_func=la
        max_min = 12 if period <= 4 else 5
        minutes = st.number_input("Minutes left in period", 0, max_min, min(6, m
        seconds = st.number_input("Seconds", 0, 59, 0) if minutes < max_min else
    raw = pd.DataFrame([{"LOC_X": x_ft * 10, "LOC_Y": y_ft * 10, "ACTION_TYPE":
                         "MINUTES_REMAINING": minutes, "SECONDS_REMAINING": seco
    feats = engineer(raw).iloc[0]
    xfg = float(model.predict_proba(raw)[0, 1])
    value = 3 if feats["is_three"] else 2
    with right:
        fig, ax = plt.subplots(figsize=(4, 4))
        court(ax); ax.scatter([x_ft * 10], [y_ft * 10], s=120, c="#E4572E", zord
        st.pyplot(fig)
    c1, c2, c3 = st.columns(3)
    c1.metric("Expected FG%", f"{xfg:.1%}")
    c2.metric("Shot value", f"{value} pts" + (" (corner)" if feats["is_corner_th
```

```
    c3.metric("Expected points", f"{xfg * value:.2f}")
    st.caption(f"Distance {feats['dist_ft']:.1f} ft · {feats['abs_angle']:.0f}°
    n_support = meta["support"].get(f"{feats['action_family']}|{feats['distance_
    if n_support < 30:
        st.warning(f"Unusual shot: only {n_support} training shots of this famil
with board:
    lb = pd.read_csv(HERE / "leaderboard.csv")
    st.write("Players with 500+ attempts. **Points over expected** = actual poin
             "an average shooter would score on the same shots. The 95 % interva
             "**shrunk** pulls noisy estimates towards zero.")
    st.dataframe(lb[["PLAYER_NAME", "team", "FGA", "FG_pct", "xFG_pct", "FG_over
                     "FGOE_shrunk", "points_over_expected"]], hide_index=True)
"""
(APP / "app.py").write_text(APP_CODE.strip() + "\n", encoding="utf-8")
v = meta["versions"]
(APP / "requirements.txt").write_text(f"streamlit>=1.40\nscikit-learn=={v['sciki
compile(APP_CODE, "app.py", "exec")
print("App files:", sorted(p.name for p in APP.iterdir() if p.is_file()))
print(f"When deploying, choose Python {v['python'][:4]} so the pinned versions i
```

```
App files: ['app.py', 'leaderboard.csv', 'model.joblib', 'model_metadata.json',
'requirements.txt', 'shot_features.py']
When deploying, choose Python 3.13 so the pinned versions install and the model l
oads.
```

In [34]:
```
# Smoke test: reload exactly as the app does and score three hand-made shots
reloaded = joblib.load(APP / "model.joblib")
demo = pd.DataFrame([{"LOC_X": 0, "LOC_Y": 5, "ACTION_TYPE": "Driving Layup Shot
                      {"LOC_X": -225, "LOC_Y": 20, "ACTION_TYPE": "Jump Shot", "P
                      {"LOC_X": 30, "LOC_Y": 400, "ACTION_TYPE": "Jump Shot", "PE
                     index=["layup at the rim", "left-corner three", "half-court
demo.assign(xFG=reloaded.predict_proba(demo)[:, 1].round(3))
```

Out[34]:

	LOC_X	LOC_Y	ACTION_TYPE	PERIOD	MINUTES_REMAINING	SECONDS_REMAI
layup at the rim	0	5	Driving Layup Shot	1	8	
left-corner three	-225	20	Jump Shot	2	5	
half-court heave	30	400	Jump Shot	2	0	

Deploy (free): put `artifacts/app/` in your GitHub repo → share.streamlit.io → *Create app* → choose `semma/artifacts/app/app.py`; under *Advanced settings* pick the Python version printed above. `shot_features.py` must stay next to `app.py` (it is the score code the model was trained with). All library versions are pinned in `requirements.txt`.

Guard-rails in the app:

- the shot-type list offers only the 44 labels with at least 50 training shots;
- the clock cannot exceed 12:00 (5:00 in overtime);

- the app warns when a shot family / distance combination had fewer than 30 training shots, e.g. a dunk from 25 ft.

Monitoring:

- *each season, compare league-wide mean xFG% with actual FG% (calibration in the large), and re-check calibration by shot zone and shot family;*
- *watch for new or retired* `ACTION_TYPE` *labels;*
- *re-train if a gap exceeds ~1 percentage point overall or ~3 points in a major segment (e.g. after rule changes or new shot trends).*

Appendix — SAS Enterprise Miner demo

SAS Enterprise Miner is available through ***SAS OnDemand for Academics*** *when your professor registers the class (SAS Studio alone is free to individuals but has no Enterprise Miner). The cell below exports the exact modeling sample, so the SAS flow uses the same data.*

In [35]:
```
# Export for SAS Enterprise Miner: one file per partition (EM: one Input Data so
# Train / Validate / Test, so EM uses this game-grouped split), plus the 2024-25
# EVENT_TYPE (the target in disguise) is removed.
part = pd.Series("train", index=sample.index)
part.iloc[idx_rest[idx_val]], part.iloc[idx_rest[idx_test]] = "validate", "test"
sas = sample.drop(columns=["GRID_TYPE", "SHOT_ATTEMPTED_FLAG", "EVENT_TYPE"]).jo
for role in ["train", "validate", "test"]:
    sas[part.to_numpy() == role].to_csv(DATA / f"nba_shots_{role}_for_sas.csv",
score_file = oot.drop(columns=["GRID_TYPE", "SHOT_ATTEMPTED_FLAG", "EVENT_TYPE",
score_file.to_csv(DATA / "nba_shots_2024_25_for_sas_scoring.csv", index=False)
REJECT = ["GAME_ID", "GAME_EVENT_ID", "GAME_DATE", "PLAYER_ID", "PLAYER_NAME", "
          "SHOT_ZONE_BASIC", "SHOT_ZONE_AREA", "SHOT_ZONE_RANGE", "SHOT_DISTANCE
          "LOC_X", "LOC_Y", "MINUTES_REMAINING", "SECONDS_REMAINING", "ACTION_TY
print(f"Exported train/validate/test files ({part.value_counts().to_dict()}) and
print("In EM, set the role to Rejected for:", ", ".join(REJECT))
print("Inputs:", ", ".join([c for c in sas.columns if c not in REJECT + [TARGET]
```

```
Exported train/validate/test files ({'train': 120014, 'validate': 40064, 'test':
39921}) and 219,527 2024-25 shots for scoring to artifacts\data
In EM, set the role to Rejected for: GAME_ID, GAME_EVENT_ID, GAME_DATE, PLAYER_I
D, PLAYER_NAME, TEAM_ID, TEAM_NAME, HTM, VTM, SEASON, SHOT_ZONE_BASIC, SHOT_ZONE_
AREA, SHOT_ZONE_RANGE, SHOT_DISTANCE, SHOT_TYPE, LOC_X, LOC_Y, MINUTES_REMAINING,
SECONDS_REMAINING, ACTION_TYPE
Inputs: PERIOD, dist_ft, log_dist, abs_angle, is_three, is_corner_three, secs_lef
t_period, heave, action, action_family, distance_bin
```

Rebuild the flow in Enterprise Miner:

1. **New project** → create a library → import the three files `nba_shots_train_for_sas.csv`, `nba_shots_validate_for_sas.csv` and `nba_shots_test_for_sas.csv` as three data sources.
2. **Input Data / variable roles:** `SHOT_MADE_FLAG` = Target (binary). Set to **Rejected** every column the export cell lists: IDs, names, dates, teams, `SEASON`, the raw zone /

distance / shot-type columns and the raw location, clock and action fields, which the engineered columns replace (same reasons as M.1). The engineered columns are Input. `EVENT_TYPE` (the target in disguise) is already removed from the export.

3. **Sample:** give the three data sources the data roles **Train**, **Validate** and **Test**, so EM uses the same game-grouped 60/20/20 split as this notebook and its results are directly comparable. (A Data Partition node would split rows at random, so shots from one game would land in several partitions.)
4. **Explore:** StatExplore, MultiPlot, Variable Selection, Cluster (on `LOC_X`, `LOC_Y`).
5. **Modify:** Replacement (rare `action` levels, or use `action_family`), Transform Variables (`log_dist` is already provided), Impute.
6. **Model:** Regression, Decision Tree, Neural Network, Gradient Boosting → **Ensemble**.
7. **Assess:** Model Comparison with **average squared error** as the selection statistic. ASE is the Brier score, which ranks the models the same way as the log loss used here (A.1). Then **Score** `nba_shots_2024_25_for_sas_scoring.csv` (the 2024-25 season, also exported).
8. **Report:** screenshot the diagram and the Model Comparison results; compare them with section A.1 in your article.